Java Collections Framework

#	Scenario (Short)	Level	Target Roles
1	Unique file paths, sorted alphabetically	L1	Backend Core, Java Developer
2	Participants in registration order (allow duplicates)	L1	Backend Core, Full Stack
3	User favorites uniquely, preserve insertion order	L1	Backend Core, Web Apps
4	Ultra-fast lookups by unique numeric ID	L1	Backend Core, Enterprise Systems
5	Cache with auto-removal when keys GC'd	L2	Backend Core, Memory-Optimized Systems
6	Changing set of numbers, quick min/max	L2	Backend Core, Quant Systems
7	Unique scores, descending ranking	L2	Backend Core, Gaming
8	Millions of users with fast lookups	L3	Backend Core, Scalable Systems
9	Always highest-priority task first	L2	Backend Core, Job Scheduling
10	Unique product names, alphabetical order	L1	Backend Core, E-commerce
11	Bidirectional list with efficient insert/remove	L2	Backend Core
12	Descending unique-score leaderboard	L2	Backend Core, Gaming
13	Alphabetical set with nearest queries	L3	Backend Core, Search Systems
14	Chat messages (add/remove both ends)	L1	Backend Core, Messaging
15	Unique student names, alphabetical	L1	Backend Core, Academic Systems
16	User borrowing history, ordered	L1	Backend Core
17	Unique recommendations, unordered	L1	Backend Core
18	News article titles, publish order, no duplicates	L2	Backend Core, Content Apps

19	Products sorted by price, range queries	L2	Backend Core, E-commerce
20	Logs by timestamp with range queries	L2	Backend Core, Logging Systems
21	Notifications in arrival order, no duplicates	L1	Backend Core
22	Revisions in order, quick oldest/newest access	L1	Backend Core, Document Systems
23	Popular inventory, reorders by access (LRU)	L3	Backend Core, Cache Systems
24	Autocomplete suggestions, alphabetical unique	L2	Backend Core, Search Systems
25	Employee name→ID, sorted by name	L2	Backend Core, Enterprise HR
26	Unique post likes, preserve arrival order	L1	Backend Core, Social Media
27	Urgent support tickets prioritized	L2	Backend Core, Support Tools
28	Dynamic numbers, fast min/max	L2	Backend Core
29	Autocomplete terms + frequency	L3	Backend Core, Search Optimization
30	Dynamic leaderboard, quick top scorer	L2	Backend Core, Gaming
31	Stock prices (oldest/newest quick)	L2	Backend Core, Trading Systems
32	Fixed-size LRU cache	L3	Backend Core, Performance
33	Logs in order, lookup by ID	L2	Backend Core
34	Add to front, trim from back efficiently	L2	Backend Core
35	Chat messages, ordered with duplicates	L1	Backend Core, Messaging
36	Recent files, unique, reorder on access	L3	Backend Core
37	Multiplayer queue (front/back process)	L2	Backend Core, Game Servers
38	Music playlist with duplicates, indexed	L1	Backend Core, Media Apps
39	Recent searches, unique, in typed order	L1	Backend Core, Search Systems
40	Priority over arrival order (strict)	L2	Backend Core, Task Queues

41	Repeated borrowings, same order	L1	Backend Core
42	Unique email IDs (and optional map)	L1	Backend Core, Identity Systems
43	Count word frequencies	L1	Backend Core, Data Analysis
44	Ongoing rides, frequent position access	L1	Backend Core, Mobility
45	Frequent insert/remove anywhere	L2	Backend Core
46	Undo/redo with bidirectional edits	L2	Backend Core, Productivity Apps
47	Unique blog tags, order irrelevant	L1	Backend Core, CMS
48	Recent searches unique, bump on reuse	L2	Backend Core, Search Optimization
49	Ascending student scores, range queries	L2	Backend Core, Academic
50	Fast dictionary word→meaning lookups	L1	Backend Core, NLP Tools
51	Preserve insertion order, unique keys	L2	Backend Core
52	Flights sorted by departure time	L2	Backend Core, Travel
53	Strict first-come, first-served queue	L1	Backend Core
54	Background jobs, priority with FIFO ties	L3	Backend Core, Distributed Systems
55	Messaging buffer push/pop both ends	L2	Backend Core, Chat Systems
56	Bidirectional contact browsing	L2	Backend Core
57	Remove safely while iterating	L1	Backend Core
58	Update/insert while iterating	L2	Backend Core
59	Natural order by numeric ID	L2	Backend Core
60	Sort products by price or name dynamically	L2	Backend Core, E-commerce
61	Sort movies by year then title	L2	Backend Core
62	Sort leaderboard descending	L2	Backend Core, Gaming
63	Check if name exists in sorted list	L2	Backend Core

64	Shuffle playlist randomly	L1	Backend Core
65	Count transactions > ₹1000 using Streams	L2	Backend Core
66	Get top 5 highest-rated movies	L2	Backend Core
67	Remove empty strings from queries	L1	Backend Core
68	Thread-safe map with atomic updates	L3	Backend Core, Concurrency
69	Read-heavy list, infrequent writes	L3	Backend Core, Real-Time Systems
70	Bounded blocking queue with backpressure	L3	Backend Core, Producer-Consumer Systems
71	Blocking priority queue for multi-threaded jobs	L3	Backend Core, Concurrency
72	Queue for delayed tasks	L3	Backend Core, Scheduler Systems
73	Concurrent sorted map with range queries	L3	Backend Core, Concurrent Systems
74	Zero-buffer direct handoff queue	L3	Backend Core, Reactive Systems
75	Lock-free MPMC event bus	L4	Backend Core, High-Throughput Systems
76	Memory-sensitive cache (GC reclaim)	L4	Backend Core, Performance-Oriented Systems

Scenario 1: Document indexing by file path

A document management tool needs to keep track of document locations by their file paths. The constraints are: no duplicate paths should be stored, and all paths should stay sorted alphabetically to allow efficient searching and retrieval.

Answer

For this use case, the right choice is TreeSet.

- It stores only **unique elements** (duplicate file paths won't be inserted).
- It maintains elements in **sorted order** automatically — for strings, this means alphabetical order, which aligns perfectly with the requirement.
- It offers efficient operations: insertion, removal, and lookup all happen in O(log n).

Code

```
import java.util.TreeSet;

public class DocumentIndexer {
    public static void main(String[] args) {
        TreeSet<String> documentPaths = new TreeSet<>();

        documentPaths.add("/work/research-paper.pdf");
        documentPaths.add("/photos/vacation.jpg");
        documentPaths.add("/work/research-paper.pdf"); // Duplicate, ignored
        documentPaths.add("/music/song.mp3");
        documentPaths.add("/docs/assignment.docx");

        System.out.println("Indexed document paths:");
        documentPaths.forEach(System.out::println);
    }
}
```

Output (alphabetically sorted, no duplicates):

/docs/assignment.docx
/music/song.mp3
/photos/vacation.jpg
/work/research-paper.pdf

Reasoning to interviewer

- Since we need both **uniqueness** and **sorted order**, TreeSet fits better than HashSet or ArrayList.
- HashSet ensures uniqueness but doesn't maintain order.

- ArrayList maintains order of insertion but allows duplicates unless explicitly checked.
- With TreeSet, both requirements are satisfied naturally, and operations like ceiling, floor, and subSet can support range-based queries, which are very handy in search systems.

Future scope

- If the number of file paths grows very large, we might consider ConcurrentSkipListSet for thread-safe operations in a multi-threaded environment.
- For case-insensitive alphabetical ordering, we can pass a custom Comparator to TreeSet.
- If we want persistence, these file paths could be stored in a database with a unique index, while still using TreeSet in-memory for caching and fast lookups.

Scenario 2: Event registration list

We are building a registration system where participants sign up with their names. The requirements are:

- Preserve the **order of registration**.
- Allow **duplicate names** since multiple people may share the same name.
- Handle frequent additions and occasional removals.

Answer

The best fit here is ArrayList.

- It preserves insertion order, so participants are listed in the sequence they registered.
- It allows duplicates, so multiple “John” entries are valid.
- Adding new names (especially at the end) is efficient.
- Removing from the middle is a bit costly, but since removals are occasional, the trade-off is acceptable.

Code

```
import java.util.ArrayList;

public class EventRegistration {
    public static void main(String[] args) {
        ArrayList<String> participantNames = new ArrayList<>();

        participantNames.add("John");
        participantNames.add("Emma");
        participantNames.add("John");  // duplicate allowed
        participantNames.add("Sophia");

        System.out.println("Registered Participants: " + participantNames);

        // Removing a participant
        participantNames.remove("Emma");
        System.out.println("After removal: " + participantNames);
    }
}
```

Reasoning to interviewer

- ArrayList was chosen because order matters, duplicates are allowed, and access by index is very fast (O(1)).

- If removals were very frequent (say, hundreds of participants cancel often), a LinkedList might be considered since it handles removals more efficiently. But in most event systems, sign-ups far outweigh cancellations, so ArrayList remains the practical choice.
- It also integrates well with features like displaying registration order directly in a UI without any extra sorting logic.

Future scope

- For thread-safe concurrent registration, CopyOnWriteArrayList or synchronization can be added.
- Pagination or filtering can be added easily since ArrayList supports random access.
- Later, participant details could be represented as an object instead of just a name for richer functionality (email, ticket number, etc.).

Scenario 3: User favorite products

We are building a platform where each user can add products to their favorites list. The requirements are:

- Each product should appear **only once**.
- The **order of addition** should be preserved, since users expect to see favorites in the sequence they selected them.

Answer

The right choice here is LinkedHashSet.

- It prevents duplicates (no product appears twice).
- It preserves the insertion order, unlike HashSet.
- It provides reasonably fast operations for add, remove, and contains (all O(1) on average).

Code

```
import java.util.LinkedHashSet;

public class UserFavorites {
    public static void main(String[] args) {
        LinkedHashSet<String> favorites = new LinkedHashSet<>();

        favorites.add("Laptop");
        favorites.add("Smartphone");
        favorites.add("Laptop"); // duplicate ignored
        favorites.add("Headphones");

        System.out.println("Favorite Products: " + favorites);
    }
}
```

Reasoning to interviewer

- A HashSet would ensure uniqueness but not order. A List would preserve order but allow duplicates. LinkedHashSet is the sweet spot, giving both properties at once.
- This makes it ideal for favorites or playlists where users expect both uniqueness and order preservation.

- Internally, LinkedHashSet uses a hash table + a linked list to maintain insertion order.

Future scope

- If we need ordering by product popularity or last-accessed, we might switch to LinkedHashMap or a priority structure.
- If favorites need to be stored per user in bulk, a Map<User, LinkedHashSet<Product>> can be used.
- For persistence, the uniqueness constraint can be enforced with a DB unique index, while still using LinkedHashSet in memory for fast user interactions.

Scenario 4: Fast lookup of user profiles by numeric ID

We're storing user profiles keyed by a unique numeric ID (e.g., 101, 102, …). The core needs are: very fast retrieval by ID, scalability to millions of users, and performance unaffected by insertion order.

Answer

I'd use a HashMap<Integer, User>.

• It gives O(1) average-time put/get, exactly what we want for ID → profile lookups.

• It's unordered, so performance doesn't depend on how or when users were added.

• It scales well; we can tune initial capacity/load factor to reduce rehashing for large datasets.

• Collisions are handled efficiently (Java 8+ can treeify buckets), so performance remains stable even with skewed keys.

Code

```
import java.util.HashMap;
import java.util.Map;

class User {
    private final int id;
    private final String name;
    private final String email;

    User(int id, String name, String email) {
        this.id = id; this.name = name; this.email = email;
    }

    public int getId() { return id; }
    public String getName() { return name; }
    public String getEmail() { return email; }

    @Override public String toString() {
        return "%d: %s <%s>".formatted(id, name, email);
    }
}

public class UserProfileManager {
    private final Map<Integer, User> byId;

    public UserProfileManager(int expectedUsers) {
        // Pre-size to reduce rehash operations for large N
        int initialCapacity = (int) (expectedUsers / 0.75f) + 1;
```

```
        this.byId = new HashMap<>(initialCapacity, 0.75f);
    }

    public void add(User u) { byId.put(u.getId(), u); }
    public User get(int id)  { return byId.get(id); }
    public User remove(int id) { return byId.remove(id); }

    public static void main(String[] args) {
        UserProfileManager mgr = new UserProfileManager(1_000_000);
        mgr.add(new User(101, "Maria", "maria@example.com"));
        mgr.add(new User(102, "David", "david@example.com"));

        System.out.println("User 101 -> " + mgr.get(101));
    }
}
```

Reasoning to interviewer

- The hot path is ID-based lookup; HashMap is designed for that. A TreeMap would give O(log n) and adds ordering we don't need. A List/array would force linear scans or manual indexing.
- Pre-sizing the map minimizes rehashing during growth. With very large user sets, that's a meaningful win.
- If IDs are long or dense, the same choice holds: HashMap<Long, User>. If memory pressure from boxing is a concern, specialized primitive maps (e.g., fastutil's Int2ObjectOpenHashMap) can remove Integer boxing overhead.
- We avoid relying on iteration order entirely, so insertion order has no performance implications.

Future scope

- Concurrency: switch to ConcurrentHashMap for multi-threaded reads/writes.
- Serialization/persistence: wrap with a DAO/repository; enforce uniqueness with a DB primary key on user_id.
- Memory tuning: adjust load factor for fewer collisions vs. memory overhead; consider primitive collections when IDs are primitives and counts are huge.
- Sharding: if the map becomes too big for a single node, shard by hash of user ID.

Scenario 5: Memory-friendly cache where keys shouldn't prevent GC

We need a cache in which entries disappear automatically when keys are no longer referenced elsewhere. Keys should not keep objects alive.

Answer

I'd use a WeakHashMap<K, V>.

• Keys are held with weak references; if the application drops all strong refs to a key, the GC can reclaim it, and the map entry evaporates.

• This avoids memory leaks for "associate metadata with an object" patterns (e.g., session objects, UI component metadata).

• Values are held strongly by default, which is fine—the key's reachability governs entry lifetime.

Code

```
import java.util.Map;
import java.util.WeakHashMap;

public class MemoryFriendlyCache {
    public static void main(String[] args) throws InterruptedException {
        Map<Object, String> cache = new WeakHashMap<>();

        Object sessionKey = new Object();           // not a literal; no global strong ref
        cache.put(sessionKey, "Cached Info");

        System.out.println("Before GC: " + cache.size()); // 1

        // Drop the last strong reference to the key
        sessionKey = null;

        // Hint GC; entry becomes eligible for removal because key is only weakly
referenced
        System.gc();
        Thread.sleep(100); // give GC a moment (demo only)

        System.out.println("After GC: " + cache.size());  // usually 0
    }
}
```

Reasoning to interviewer:

• WeakHashMap solves the classic “attach cache to object without leaking memory” problem. When the object (key) is no longer used elsewhere, the entry disappears without us writing cleanup code.
• Important nuance: string literals or other globally referenced keys won’t be collected—so for demos/tests, use new String(...) or non-interned/unique key objects.
• The map is not a reliability boundary: entries may vanish at any time after keys lose strong reachability. Don’t base correctness on size() or assume permanence.
• If we need eviction policy (LRU/LFU) rather than GC-driven cleanup, an access-ordered LinkedHashMap or a library cache (Caffeine/Guava) is more appropriate. If values, not keys, should be memory-sensitive, consider wrapping values in SoftReference (with caveats) or use Caffeine which handles this robustly.

Future scope:

• Thread-safety: wrap with Collections.synchronizedMap or use external locking; for high concurrency and policies, prefer Caffeine (time/size-based eviction, stats).
• Observability: add ReferenceQueue handling if we need callbacks/metrics on eviction (WeakHashMap uses one internally, but custom refs allow hooks).
• Mixed strategy: combine LinkedHashMap (access-order) for LRU behavior with weak keys only when the associated object’s lifecycle should own the cache entry.
• Production-ready caching: use Caffeine for bounded size + time-based expiry + weak/soft reference options, hit-rate metrics, and high concurrency.

Scenario 6: Tracking min and max in a dynamic dataset

We're building a data processing system where numbers are added or removed frequently. After every update, the system must quickly answer: what's the smallest value? what's the largest?

Answer

The natural choice is TreeSet<Integer>.

- It keeps numbers sorted automatically.
- first() gives the minimum and last() gives the maximum in O(1) time.
- Insertions and deletions are O(log n), which is efficient for a continuously changing dataset.
- Unlike manually sorting a list each time, the structure maintains order incrementally, which is more efficient.

Code

```
import java.util.TreeSet;

public class DynamicNumberSet {
    public static void main(String[] args) {
        TreeSet<Integer> numberSet = new TreeSet<>();

        numberSet.add(70);
        numberSet.add(10);
        numberSet.add(90);

        System.out.println("Minimum number: " + numberSet.first());
        System.out.println("Maximum number: " + numberSet.last());

        numberSet.remove(10);
        System.out.println("After removal, new min: " + numberSet.first());
    }
}
```

Reasoning to interviewer

- With a HashSet we'd lose ordering, and finding min/max would require scanning the entire set (O(n)).
- With a PriorityQueue, we could get min efficiently, but not max (or vice versa, depending on comparator). A single PriorityQueue doesn't solve both.

- TreeSet is balanced for this use case: both ends are accessible instantly, and the sorted property is preserved automatically.
- The trade-off is O(log n) insertion/removal, but this is acceptable and scalable compared to repeated re-sorting.

Future scope

- If we need to support duplicate numbers, switch to TreeMap<Integer, Integer> (value counts) or PriorityQueue pairs.
- For very large datasets, consider specialized libraries (like fastutil or Trove) for primitive collections to avoid boxing overhead.
- For concurrent data processing, ConcurrentSkipListSet is a thread-safe alternative.

Scenario 7: Student ranking in descending order with uniqueness

We need to store student scores where:

- Scores must be unique.
- They should be displayed in descending order (highest score first).

Answer

A TreeSet with Comparator.reverseOrder() is the right fit.

- Ensures uniqueness by design.
- Maintains a sorted order, but reversed to show highest first.
- Provides efficient add/remove/lookups in O(log n).
- Iterating directly gives scores ranked top-down.

Code

```
import java.util.Comparator;
import java.util.TreeSet;

public class ScoreRanking {
    public static void main(String[] args) {
        TreeSet<Integer> scores = new TreeSet<>(Comparator.reverseOrder());

        scores.add(92);
        scores.add(88);
        scores.add(95);
        scores.add(92); // duplicate ignored

        System.out.println("Scores in descending order: " + scores);

        System.out.println("Top scorer: " + scores.first());
        System.out.println("Lowest scorer: " + scores.last());
    }
}
```

Reasoning to interviewer

- HashSet or LinkedHashSet would allow uniqueness, but no descending order.
- A List would need sorting after every insert, which is costly.

- With TreeSet + reverse comparator, order is enforced on insertion itself. Iterating gives a ranked scoreboard instantly.
- Extra benefit: we can easily fetch ranges like “all scores above 90” with headSet or tailSet.

Future scope

- If we also need to associate scores with student names, we can use a TreeMap<Integer, String> with a reverse comparator on keys.
- If multiple students can share the same score, switch to TreeMap<Integer, List<String>>.
- For concurrent ranking systems (e.g., online exams), use ConcurrentSkipListSet with a reverse comparator.

Scenario 8: Fast user lookups by unique numerical ID

We're designing a system where each user is identified by a unique numeric ID (like 1001, 1002). The requirements are:

- Support millions of users.
- Very fast lookups by ID.
- Lookup performance should not depend on insertion order.

Answer

The best fit here is HashMap<Integer, User>.

- Average O(1) for put() and get() ensures fast performance even at scale.
- IDs are unique, so keys map cleanly to a single user record.
- Unlike lists or arrays, we don't rely on ID being continuous or small; hashing works regardless.
- Since order isn't required, HashMap avoids unnecessary overhead compared to ordered structures like TreeMap.

Code

```
import java.util.HashMap;
import java.util.Map;

class User {
    private final int id;
    private final String name;

    User(int id, String name) {
        this.id = id; this.name = name;
    }

    public int getId() { return id; }
    public String getName() { return name; }

    @Override
    public String toString() {
        return id + ": " + name;
    }
}

public class UserManager {
    public static void main(String[] args) {
```

```
        Map<Integer, User> users = new HashMap<>();

        users.put(1001, new User(1001, "Emma"));
        users.put(1002, new User(1002, "Liam"));

        System.out.println("User with ID 1001: " + users.get(1001));
    }
}
```

Reasoning to interviewer

- HashMap ensures predictable performance regardless of order of insertion, unlike a TreeMap which is O(log n) due to sorting.
- Using IDs as keys leverages their uniqueness.
- If insertion order ever mattered, we could use LinkedHashMap. But here performance is more critical than order.
- If thread safety is needed, ConcurrentHashMap is the natural extension.

Future scope

- For distributed systems, sharding user data across multiple nodes keyed by hash of ID.
- For persistence, enforce uniqueness via a DB primary key while using HashMap in memory as a cache.
- Use primitive collections libraries to reduce memory overhead if billions of users are expected.

Scenario 9: Task scheduling with priorities

We need a system that always processes the highest-priority task first. Priorities are numeric, with smaller numbers representing higher importance.

Answer

The right collection is PriorityQueue.

- It's a heap-backed structure, always exposing the "smallest" (or "highest priority") element at the head.
- It automatically reorders tasks as new ones arrive.
- Operations like add() and poll() are O(log n), which is efficient for real-time task scheduling.
- If we want descending order (largest number as highest priority), we can supply a custom comparator.

Code

```
import java.util.PriorityQueue;

class Task {
    private final int priority;
    private final String description;

    Task(int priority, String description) {
        this.priority = priority;
        this.description = description;
    }

    public int getPriority() { return priority; }
    public String getDescription() { return description; }

    @Override
    public String toString() {
        return "Priority " + priority + ": " + description;
    }
}

public class TaskScheduler {
    public static void main(String[] args) {
        PriorityQueue<Task> tasks = new PriorityQueue<>(
            (t1, t2) -> Integer.compare(t1.getPriority(), t2.getPriority())
        );

        tasks.add(new Task(10, "Low-priority cleanup"));
```

```
        tasks.add(new Task(1, "Process payment"));
        tasks.add(new Task(5, "Send email notification"));

        while (!tasks.isEmpty()) {
            System.out.println("Processing: " + tasks.poll());
        }
    }
}
```

Reasoning to interviewer

- A TreeSet could order tasks but would disallow duplicate priorities; PriorityQueue supports duplicates naturally.
- With a LinkedList or ArrayList, we'd have to sort manually each time, which is inefficient.
- PriorityQueue rebalances automatically, ensuring the head is always the next highest-priority task.
- In this example, "Process payment" (priority 1) comes before "Send email notification" (priority 5), regardless of insertion order.

Future scope

- Use DelayQueue if tasks should also consider scheduled execution time.
- Wrap with ExecutorService for concurrent execution of tasks.
- For custom business rules, supply different comparators (e.g., tie-breaking on creation time).
- For massive systems like OS schedulers, a multi-level priority queue (bucketed) may be more efficient.

Scenario 10: Inventory product names in sorted order

We're building an inventory system where product names must be stored uniquely and frequently retrieved in alphabetical order, especially for reports.

Answer

The right collection here is TreeSet<String>.

- It ensures uniqueness, so no duplicate product entries.
- It automatically sorts product names in their natural order (alphabetical for strings).
- It provides efficient O(log n) operations for insert, search, and retrieval.
- Ideal for generating sorted inventory reports without needing extra sorting logic.

Code

```
import java.util.TreeSet;

public class ProductInventory {
    public static void main(String[] args) {
        TreeSet<String> productNames = new TreeSet<>();

        productNames.add("Tablet");
        productNames.add("Smartphone");
        productNames.add("Laptop");
        productNames.add("Smartphone"); // duplicate ignored

        System.out.println("Products sorted alphabetically: " + productNames);
    }
}
```

Reasoning to interviewer

- A HashSet would give uniqueness but not order, requiring explicit sorting each time.
- An ArrayList could store items, but duplicates must be handled manually, and sorting repeatedly would be costly.
- With TreeSet, uniqueness and order are maintained automatically, reducing boilerplate and improving performance.

- For large datasets, it scales well since Red-Black tree operations remain logarithmic.

Future scope

- Use Comparator if we want case-insensitive sorting or custom ordering (e.g., by product category then name).
- For concurrent inventory updates, ConcurrentSkipListSet is a thread-safe alternative.
- If product details (price, stock) are also needed, a TreeMap<String, Product> could map name → product object while keeping names sorted.

Scenario 11: Contact list with two-way navigation and frequent edits

We're building a contact management app where users can scroll forwards and backwards through contacts. We also need efficient insertion and removal at arbitrary positions.

Answer

The right choice is LinkedList<String>.

- It supports a ListIterator, allowing bi-directional traversal.
- Insertions and deletions at any position are efficient since only node references are updated.
- Unlike ArrayList, where middle insertions/removals require shifting elements, LinkedList avoids that overhead.
- It's suitable for apps where navigation and dynamic modifications matter more than random index-based lookups.

Code

```
import java.util.LinkedList;
import java.util.ListIterator;

public class ContactManager {
    public static void main(String[] args) {
        LinkedList<String> contacts = new LinkedList<>();

        contacts.add("John");
        contacts.add("Sara");
        contacts.add("Alex");

        // Traverse forward
        ListIterator<String> iterator = contacts.listIterator();
        System.out.println("Forward navigation:");
        while (iterator.hasNext()) {
            System.out.println("Contact: " + iterator.next());
        }

        // Traverse backward
        System.out.println("Backward navigation:");
        while (iterator.hasPrevious()) {
            System.out.println("Contact: " + iterator.previous());
        }
    }
}
```

Reasoning to interviewer

- ArrayList gives O(1) random access but O(n) for middle insertions/deletions, which is not efficient here.
- LinkedList shines in scenarios where sequential navigation and frequent structural changes are needed.
- ListIterator is key here: it provides smooth two-way navigation that a simple Iterator doesn’t.
- The trade-off is slower random access (O(n)), but in a contact browsing app, sequential navigation is the primary use case.

Future scope

- If we need sorting by name or last interaction, a TreeMap or sorted structure could complement LinkedList.
- For very large datasets, indexing or hybrid data structures (like skip lists) could improve search efficiency.
- If concurrent modifications are expected (e.g., shared address book), consider CopyOnWriteArrayList for read-heavy scenarios, though it trades off insertion performance.

Scenario 12: Gaming leaderboard with unique scores in descending order

We're building a leaderboard that must:

- Keep scores **unique**.
- Always display scores from **highest to lowest**.
- Allow quick access to top scores.

Answer

The best fit here is TreeSet<Integer> with a Comparator.reverseOrder().

- TreeSet guarantees uniqueness automatically.
- Passing a reverse comparator ensures scores are ordered from highest to lowest.
- Retrieval of top score is just first(), and iteration gives a ranked leaderboard directly.
- Insertion and lookup are O(log n), which is acceptable for this use case.

Code

```
import java.util.Comparator;
import java.util.TreeSet;

public class GameLeaderboard {
    public static void main(String[] args) {
        TreeSet<Integer> playerScores = new
TreeSet<>(Comparator.reverseOrder());

        playerScores.add(1200);
        playerScores.add(850);
        playerScores.add(950);
        playerScores.add(1200); // duplicate ignored

        System.out.println("Leaderboard Scores: " + playerScores);

        System.out.println("Top Score: " + playerScores.first());
        System.out.println("Lowest Score: " + playerScores.last());
    }
}
```

Reasoning to interviewer

- If we used PriorityQueue, we could get the highest score but not maintain a full ordered view for iteration.
- HashSet or ArrayList would need extra sorting each time we display the leaderboard.
- TreeSet with descending order keeps the leaderboard continuously sorted without manual intervention.
- By disallowing duplicates, we enforce one unique score per player, which matches many leaderboard models.

Future scope

- If players can tie with the same score, we'd need TreeMap<Integer, List<Player>> where score maps to multiple players.
- For very large leaderboards with frequent updates, we could shard scores into ranges or use a database index.
- If we need concurrent updates from multiple game servers, ConcurrentSkipListSet with reverse order provides thread safety.

Scenario 13: Directory with nearest name lookups

We're building a directory that stores names alphabetically. Requirement:

- Keep names sorted.
- Find the closest match above or below a given name, useful for features like auto-suggestion.

Answer

The right structure is NavigableSet<String>, typically implemented using TreeSet.

- It maintains alphabetical order automatically.
- Methods like higher(), lower(), ceiling(), and floor() allow efficient retrieval of the nearest names around a target.
- Perfect for search bars or directory apps where users type partial names.

Code

```
import java.util.NavigableSet;
import java.util.TreeSet;

public class NameDirectory {
    public static void main(String[] args) {
        NavigableSet<String> nameSet = new TreeSet<>();

        nameSet.add("Emma");
        nameSet.add("Liam");
        nameSet.add("Olivia");
        nameSet.add("Sophia");

        System.out.println("Name just after Emma: " + nameSet.higher("Emma"));   //
Liam
        System.out.println("Name just before Olivia: " + nameSet.lower("Olivia")); //
Liam
        System.out.println("Ceiling for Noah: " + nameSet.ceiling("Noah"));       //
Olivia
        System.out.println("Floor for Noah: " + nameSet.floor("Noah"));           // Liam
    }
}
```

Reasoning to interviewer

- A plain TreeSet gives sorting but doesn't expose navigation methods directly; NavigableSet enhances it with nearest-match queries.
- A HashSet can't guarantee order, so it's not suitable for alphabetical searching.
- Lists would require sorting and binary search manually. With NavigableSet, this logic is built in and efficient.
- Complexity is O(log n) for insertions/lookups, which scales well.

Future scope

- For case-insensitive directories, use a custom comparator (String.CASE_INSENSITIVE_ORDER).
- For large-scale directories (millions of names), indexes in a DB or trie-based structures may complement TreeSet.
- If auto-complete requires prefix search, combining TreeSet with methods like subSet(prefix, nextPrefix) is powerful.

Scenario 14: Chat messages with add/remove from both ends

We're building a chat app where messages must be stored in arrival order, but also support removing from both the **oldest end** (when history is trimmed) and the **newest end** (when the latest message is deleted).

Answer

The right choice is a Deque (double-ended queue).

- It supports efficient O(1) insertion and removal at both ends.
- ArrayDeque is usually faster than LinkedList due to better cache performance, so it's preferred for most cases.
- The API is expressive: addFirst, addLast, removeFirst, removeLast directly model our chat operations.

Code

```
import java.util.Deque;
import java.util.ArrayDeque;

public class MessageQueue {
    public static void main(String[] args) {
        Deque<String> messages = new ArrayDeque<>();

        // Messages arrive
        messages.addLast("First message");
        messages.addLast("Second message");

        // Insert urgent message at front
        messages.addFirst("Urgent message");

        System.out.println("Processing: " + messages.removeFirst()); // Urgent
message
        System.out.println("Next message: " + messages.removeLast()); // Second
message
    }
}
```

Reasoning to interviewer

- A plain Queue only supports operations at one end; a Deque extends this for bi-directional flexibility.
- ArrayDeque avoids the overhead of linked nodes and is not capacity-constrained like Stack/Vector.

- Unlike Stack, it can pop from both ends, which fits message deletion from newest or oldest.
- Complexity for all these operations is O(1), making it ideal for a high-throughput messaging system.

Future scope

- For thread-safe messaging between producer and consumer threads, ConcurrentLinkedDeque is a good alternative.
- If message expiry times are needed, pairing Deque with a scheduler or timestamp metadata helps auto-prune old messages.
- For persistence (chat history), messages can be flushed from deque to a database once size or time thresholds are exceeded.

Scenario 15: Classroom registry with alphabetical names, no duplicates

We're designing a school system where:

- Student names must be kept in **alphabetical order**.
- Each student should appear **only once** in a class.

Answer

The right collection is TreeSet<String>.

- It ensures uniqueness (no duplicates).
- It sorts student names automatically in natural order (alphabetical).
- This makes tasks like generating attendance or reporting seamless without manual sorting.

Code

```
import java.util.TreeSet;

public class ClassroomRegistry {
    public static void main(String[] args) {
        TreeSet<String> enrolledStudents = new TreeSet<>();

        enrolledStudents.add("Michael");
        enrolledStudents.add("Anna");
        enrolledStudents.add("Sophia");
        enrolledStudents.add("Michael"); // duplicate ignored

        System.out.println("Students (alphabetical): " + enrolledStudents);
    }
}
```

Reasoning to interviewer

- A HashSet ensures uniqueness but doesn't sort, requiring extra sorting for reports.
- A List allows duplicates and needs manual sorting and filtering.

- With TreeSet, both uniqueness and order are enforced as elements are added.
- Operations like headSet and tailSet can also easily answer queries like “all students before ‘N’ alphabetically.”

Future scope

- For case-insensitive storage, use a custom comparator (String.CASE_INSENSITIVE_ORDER).
- If we want to store more details (age, grade) along with name, a TreeMap<String, Student> could be used.
- For very large classes with concurrent updates, ConcurrentSkipListSet provides thread-safe operations while maintaining order.

Scenario 16: Borrowing history in a library system

We need to track books borrowed by each user in the **exact sequence** they were taken out. This history must reflect the borrowing order and allow updates when books are borrowed or returned.

Answer

The best choice is LinkedList<String>.

- It preserves insertion order, so borrowing history is maintained correctly.
- Adding newly borrowed books at the end is efficient.
- Removing returned books from either end (or even the middle) doesn't involve shifting elements like ArrayList would.
- It integrates well with iterators for sequential traversal of borrowing history.

Code

```
import java.util.LinkedList;

public class BorrowHistory {
    public static void main(String[] args) {
        LinkedList<String> booksBorrowed = new LinkedList<>();

        booksBorrowed.add("Pride and Prejudice");
        booksBorrowed.add("To Kill a Mockingbird");
        booksBorrowed.add("1984");

        System.out.println("Borrowing History: " + booksBorrowed);

        // Returning the first borrowed book
        booksBorrowed.removeFirst();
        System.out.println("After return: " + booksBorrowed);
    }
}
```

Reasoning to interviewer

- A LinkedList is a natural fit when maintaining ordered histories with frequent insertions/removals.
- ArrayList would preserve order too but is inefficient for removals in the middle or front due to shifting costs.

- Here, we care about maintaining history order and supporting efficient updates, which is exactly what LinkedList excels at.

Future scope

- To associate borrowing history per user, use a Map<User, LinkedList<Book>>.
- If we want quick lookups to check if a user already borrowed a specific book, combine with a HashSet.
- For long-term persistence, borrowing history could be periodically synced with a database while keeping recent activity in memory.

Scenario 17: Unique product recommendations in an e-commerce platform

We want to show product recommendations based on recently viewed items, but **no duplicates** should appear.

Answer

The right choice is a HashSet<String>.

- It ensures uniqueness automatically—duplicates are ignored.
- It provides average O(1) insertion and lookup, which is crucial for a high-performance recommendation engine.
- For this use case, we don't care about order, only that duplicates don't appear.

Code

```
import java.util.HashSet;

public class RecommendationEngine {
    public static void main(String[] args) {
        HashSet<String> recommendedItems = new HashSet<>();

        recommendedItems.add("Headphones");
        recommendedItems.add("Smartwatch");
        recommendedItems.add("Headphones"); // duplicate ignored
        recommendedItems.add("Laptop");

        System.out.println("Unique Product Recommendations: " +
recommendedItems);
    }
}
```

Reasoning to interviewer

- A List would allow duplicates unless we explicitly filtered them, which adds overhead.
- A TreeSet would also ensure uniqueness but imposes a sorted order, which is unnecessary here.
- HashSet gives the most efficient insertions and checks for duplicates, aligning with the goal of keeping recommendations unique and performant.

Future scope

- If we need to preserve the order in which products were viewed, LinkedHashSet would be a better choice.
- For ranking recommendations by relevance or timestamp, TreeSet with a comparator could be used.
- If the system is multi-threaded and frequently updated, ConcurrentHashMap-based sets (ConcurrentHashMap.newKeySet()) can be used for thread safety.

Scenario 18: News articles in published order without duplicates

We need to store articles exactly in the order they were published while preventing duplicate titles.

Answer

The best collection is LinkedHashSet<String>.

- It preserves insertion order, which matches the publish sequence.
- It guarantees uniqueness, so duplicate articles (same title) won't be added again.
- It provides O(1) average-time operations for add, remove, and contains, which is efficient for a constantly updating news feed.

Code

```
import java.util.LinkedHashSet;

public class NewsFeed {
    public static void main(String[] args) {
        LinkedHashSet<String> newsArticles = new LinkedHashSet<>();

        newsArticles.add("Java 17 Official Release");
        newsArticles.add("AI Trends for 2025");
        newsArticles.add("Cloud Computing in 2025");
        newsArticles.add("Java 17 Official Release"); // duplicate ignored

        System.out.println("News Articles in order:");
        newsArticles.forEach(System.out::println);
    }
}
```

Reasoning to interviewer

- A HashSet would remove duplicates but wouldn't preserve publish order.
- An ArrayList would preserve order but allow duplicates, requiring extra logic to filter them out.
- LinkedHashSet combines the strengths of both: order is preserved and duplicates are automatically filtered.
- This ensures the news feed stays accurate and consistent without extra processing.

Future scope

- If articles have more data (author, date, content), we can store Article objects and override equals/hashCode on the title field to prevent duplicates.
- For paginated feeds, iteration over LinkedHashSet integrates well.
- For distributed feeds, we may sync with a database or cache layer but still use LinkedHashSet in memory for real-time ordering and uniqueness.

Scenario 19: E-commerce products sorted by price with range queries

We're building a product catalog where:

- Products must be stored by **price order**.
- We need efficient queries for ranges, like "products under $500" or "between $500 and $1000."

Answer

The right collection is TreeMap<Integer, String> (price → product).

- It keeps entries sorted by key (price).
- Provides range query methods like headMap(), tailMap(), and subMap().
- Supports ascending or descending order traversal through navigableKeySet() or descendingMap().

Code

```
import java.util.NavigableMap;
import java.util.TreeMap;

public class PriceCatalog {
    public static void main(String[] args) {
        TreeMap<Integer, String> productPrices = new TreeMap<>();

        productPrices.put(199, "Wireless Mouse");
        productPrices.put(899, "Smartphone");
        productPrices.put(499, "Bluetooth Speaker");
        productPrices.put(1299, "Laptop");

        System.out.println("All Products: " + productPrices);
        System.out.println("Products below $500: " + productPrices.headMap(500));
        System.out.println("Products $500 and above: " +
productPrices.tailMap(500));
        System.out.println("Products between $500 and $1000: " +
productPrices.subMap(500, true, 1000, true));
    }
}
```

Reasoning to interviewer

- A HashMap is fast but doesn’t keep prices sorted, so range queries would be expensive.
- A List of products could be sorted, but keeping it sorted after frequent inserts/removals is inefficient.
- TreeMap automatically maintains ordering and provides built-in methods for efficient range queries.
- This makes it a perfect fit for catalogs, pricing filters, and product browsing features.

Future scope

- To allow multiple products at the same price, store List<Product> as values instead of single strings.
- For descending price order, use productPrices.descendingMap().
- For concurrency in large catalogs, ConcurrentSkipListMap is the thread-safe alternative.
- If catalog grows huge, we can offload to a database with indexed queries but still use TreeMap in memory for caching.

Scenario 20: Log analysis with timestamp-based range queries

We’re analyzing logs from multiple servers, each entry tagged with a timestamp. Requirement:

- Store logs keyed by timestamp.
- Efficiently query logs between two points in time (e.g., last hour).

Answer

The right collection is TreeMap<Long, String>.

- Keys (timestamps) are sorted automatically.
- Range queries like subMap(start, end), headMap(), and tailMap() make it efficient to retrieve logs within specific time intervals.
- Insertions and lookups are O(log n), which scales well for large log sets.

Code

```
import java.util.TreeMap;

public class ServerLogAnalyzer {
    public static void main(String[] args) {
        TreeMap<Long, String> serverLogs = new TreeMap<>();

        serverLogs.put(1680300000000L, "Server started");
        serverLogs.put(1680303600000L, "User login");
        serverLogs.put(1680307200000L, "Error reported");

        long rangeStart = 1680300000000L;
        long rangeEnd = 1680303600000L;

        System.out.println("Logs in specified range: " +
serverLogs.subMap(rangeStart, rangeEnd));
    }
}
```

Reasoning to interviewer

- A HashMap would not preserve order, forcing us to scan all logs for a range.
- A List would require sorting before range queries, which is inefficient at scale.

- TreeMap maintains chronological order automatically and gives log slices on demand.
- With NavigableMap, we can also use floorEntry and ceilingEntry to snap to nearest timestamps.

Future scope

- If multiple logs share the same timestamp, use TreeMap<Long, List<String>>.
- For descending order (latest logs first), call descendingMap().
- For concurrent, large-scale logging, ConcurrentSkipListMap offers thread safety.
- For persistence, database indexes on timestamp can complement TreeMap in-memory caching.

Scenario 21: Notifications in order without duplicates

We're building a user notification system where:

- Notifications must be shown in **arrival order**.
- Each notification should be unique (no repeats).

Answer

The right choice is LinkedHashSet<String>.

- It preserves insertion order, so notifications appear in the exact sequence received.
- It ensures uniqueness automatically by discarding duplicates.
- Average O(1) for add and lookup, which is efficient for frequent updates.

Code

```
import java.util.LinkedHashSet;

public class UserNotificationCenter {
    public static void main(String[] args) {
        LinkedHashSet<String> userNotifications = new LinkedHashSet<>();

        userNotifications.add("Message from Alice");
        userNotifications.add("Friend request from Bob");
        userNotifications.add("Message from Alice"); // duplicate ignored

        System.out.println("User Notifications: " + userNotifications);
    }
}
```

Reasoning to interviewer

- A HashSet enforces uniqueness but doesn't preserve order.
- An ArrayList preserves order but allows duplicates unless manually filtered.
- LinkedHashSet balances both: the feed stays clean, ordered, and unique automatically.
- Perfect for user-facing feeds where duplicates degrade experience.

Future scope

- If we need notifications sorted by priority instead of arrival order, TreeSet with a custom comparator could be used.
- For per-user feeds, use a Map<User, LinkedHashSet<Notification>>.
- For distributed systems, pair it with a persistence layer (like Redis sorted sets).

Scenario 22: Collaborative document revisions in chronological order

We're building a live document editor where revisions:

- Must be stored in the order they're created.
- Should allow fast access to oldest (first draft) and newest (latest change).

Answer

The right collection is ArrayDeque<String>.

- It preserves insertion order.
- Provides O(1) access/removal at both ends (getFirst(), getLast()).
- More efficient than LinkedList due to contiguous memory and lower overhead.
- Fits perfectly for a revision history buffer where we often append and occasionally check oldest/newest entries.

Code

```
import java.util.ArrayDeque;
import java.util.Deque;

public class RevisionManager {
    public static void main(String[] args) {
        Deque<String> revisionHistory = new ArrayDeque<>();

        revisionHistory.addLast("Draft created");
        revisionHistory.addLast("Introduction added");
        revisionHistory.addLast("Spelling corrections made");

        System.out.println("Earliest revision: " + revisionHistory.getFirst());
        System.out.println("Most recent revision: " + revisionHistory.getLast());
    }
}
```

Reasoning to interviewer

- ArrayDeque is faster than LinkedList for most queue/deque operations because it avoids node allocation overhead.
- ArrayList preserves order but isn't efficient for removals at both ends.

- ArrayDeque is specifically designed for scenarios like revision logs, message queues, and undo-redo stacks.

Future scope

- Implement an undo/redo system using two ArrayDeques (one for history, one for undone actions).
- For multi-user collaboration, combine with version metadata (user, timestamp).
- For very long histories, consider trimming old entries when size exceeds a limit.
- For thread-safe collaborative editing, use ConcurrentLinkedDeque.

Scenario 23: Warehouse inventory ordered by access frequency

We're designing an inventory system where items are reordered based on how frequently they're accessed. Popular items must float to the top, making them easy to identify and retrieve.

Answer

The best option is a LinkedHashMap configured in **access-order mode** (accessOrder = true).

- Normally, LinkedHashMap maintains insertion order.
- With access-order enabled, every time an item is accessed (via get or put), it's moved to the end.
- This effectively creates an LRU (least recently used) ordering, where least-accessed items are at the front and most-accessed ones at the back.
- By iterating from the end, we can quickly reach the most popular items.

Code

```
import java.util.LinkedHashMap;
import java.util.Map;

public class PopularityInventory extends LinkedHashMap<String, Integer> {
    private final int maxCapacity;

    public PopularityInventory(int capacity) {
        super(capacity, 0.75f, true);  // 'true' enables access-order
        this.maxCapacity = capacity;
    }

    public void addOrUpdateItem(String item, int quantity) {
        put(item, quantity);
    }

    public Integer getStock(String item) {
        return get(item);  // Accessing moves it to the end
    }

    @Override
    protected boolean removeEldestEntry(Map.Entry<String, Integer> eldest) {
        return size() > maxCapacity; // optional eviction policy
    }

    public static void main(String[] args) {
        PopularityInventory inventory = new PopularityInventory(5);
```

```
        inventory.addOrUpdateItem("Monitor", 20);
        inventory.addOrUpdateItem("Keyboard", 35);
        inventory.addOrUpdateItem("Mouse", 50);

        inventory.getStock("Monitor");  // Access reorders "Monitor"

        System.out.println("Inventory (least → most accessed): " + inventory);
    }
}
```

Reasoning to interviewer

- HashMap gives fast lookups but doesn't preserve access order.
- TreeMap sorts by keys, not usage frequency.
- LinkedHashMap with access-order is ideal because it automatically reorders elements upon access, making it easy to identify popular items.
- This same technique is used internally to build simple LRU caches.

Future scope

- We could extend this to evict least-used items automatically using removeEldestEntry.
- For concurrency, a ConcurrentLinkedHashMap (from Guava or Caffeine library) is better in production.
- Popularity can also be tracked by maintaining explicit counters in addition to access order, if we want **true frequency** instead of **recent usage**.

Scenario 24: Autocomplete suggestions in alphabetical order

We're building an autocomplete system where:

- Suggestions must appear in alphabetical order.
- The list should be updated dynamically as new terms are added.
- Duplicates must not appear.

Answer

The correct choice is TreeSet<String>.

- It keeps elements sorted alphabetically by default.
- It enforces uniqueness (no duplicate suggestions).
- Insertions and lookups are O(log n), which is efficient enough for a continuously updated autocomplete system.
- The ordered nature makes it trivial to display suggestions in the right order.

Code

```
import java.util.TreeSet;

public class SearchAutocomplete {
    public static void main(String[] args) {
        TreeSet<String> suggestionSet = new TreeSet<>();

        suggestionSet.add("Apple");
        suggestionSet.add("Orange");
        suggestionSet.add("Banana");
        suggestionSet.add("Apple"); // duplicate ignored

        System.out.println("Autocomplete Suggestions: " + suggestionSet);
    }
}
```

Reasoning to interviewer

- A HashSet ensures uniqueness but gives no ordering, which is mandatory here.
- A List would allow duplicates and require explicit sorting after every update.

- With TreeSet, insertion itself maintains sorted order, so no extra processing is needed for autocomplete.

Future scope

- Use subSet(prefix, nextPrefix) for prefix-based queries (e.g., all suggestions starting with “Ap”).
- For case-insensitive autocomplete, initialize TreeSet with a custom comparator.
- For large-scale systems, a trie data structure may be more efficient for prefix-based lookups, but TreeSet works very well for moderate-scale autocomplete needs.

Scenario 25: Employee directory with alphabetical names and unique IDs

We're building an employee directory that should:

- Store employee names mapped to IDs.
- Maintain names in **alphabetical order**.
- Prevent duplicate names.
- Allow fast lookups by name.

Answer

The best fit is a TreeMap<String, Integer>.

- Keys (employee names) are unique and sorted automatically.
- Each name is mapped to its employee ID.
- get() provides O(log n) retrieval, which is efficient even as the directory grows.
- The natural ordering of keys ensures that reports or listings are alphabetically arranged by default.

Code

```
import java.util.TreeMap;

public class EmployeeDirectorySystem {
    public static void main(String[] args) {
        TreeMap<String, Integer> employeeMap = new TreeMap<>();

        employeeMap.put("Alice", 1001);
        employeeMap.put("Bob", 1002);
        employeeMap.put("Charlie", 1003);

        System.out.println("Employee Directory: " + employeeMap);
        System.out.println("ID of Alice: " + employeeMap.get("Alice"));
    }
}
```

Reasoning to interviewer

- A HashMap would give fast lookups but not alphabetical order.
- A TreeSet would maintain unique names but wouldn't store employee IDs with them.

- A TreeMap is the right balance—it guarantees alphabetical uniqueness on names and allows mapping to IDs.
- With methods like headMap and tailMap, we can also fetch employees in specific alphabetical ranges, which is useful in directories.

Future scope

- For case-insensitive directories, initialize TreeMap with a comparator (String.CASE_INSENSITIVE_ORDER).
- If employee details are richer than just IDs, the value can be a Employee object.
- For concurrent environments, ConcurrentSkipListMap is a thread-safe alternative.

Scenario 26: Social media post likes with order preserved and uniqueness enforced

We're building a feature to track which users have liked a post. Requirements:

- Each user can like the post **only once**.
- Likes must be shown in the **order they arrived**.

Answer

The best collection is LinkedHashSet<String>.

- It prevents duplicate likes (same user liking twice).
- It preserves insertion order, so likes are displayed in the sequence they were added.
- It provides O(1) average-time insertion and lookup, making it efficient for tracking engagement.

Code

```
import java.util.LinkedHashSet;

public class PostLikeTracker {
    public static void main(String[] args) {
        LinkedHashSet<String> userLikes = new LinkedHashSet<>();

        userLikes.add("UserA");
        userLikes.add("UserB");
        userLikes.add("UserA"); // duplicate ignored
        userLikes.add("UserC");

        System.out.println("Users who liked the post: " + userLikes);
    }
}
```

Reasoning to interviewer

- An ArrayList would preserve order but allow duplicates unless filtered manually.
- A HashSet prevents duplicates but doesn't preserve order.

- LinkedHashSet combines both requirements, making it ideal for user engagement features like likes or followers.

Future scope

- For large-scale feeds, likes can be sharded across servers but still maintained in LinkedHashSet per post in memory.
- If we want to store additional metadata (like time of like), we can switch to a LinkedHashMap<User, Timestamp>.
- For distributed systems, this logic can map well to database tables with unique constraints, while LinkedHashSet provides in-memory consistency.

Scenario 27: Prioritizing urgent tickets in a support system

We're building a support ticket system where tickets can be urgent or normal. The requirement is that urgent tickets **must always be handled first**, regardless of arrival order.

Answer

A PriorityQueue<ServiceTicket> with a **custom comparator** is the right approach.

- PriorityQueue always dequeues the "highest priority" element first.
- By designing a comparator that puts urgent tickets ahead of normal ones, we ensure critical issues are handled first.
- Insertions and removals are O(log n), making it efficient for real-time ticket management.

Code

```
import java.util.PriorityQueue;

class ServiceTicket {
    String issueDescription;
    boolean urgentFlag;

    ServiceTicket(String issueDescription, boolean urgentFlag) {
        this.issueDescription = issueDescription;
        this.urgentFlag = urgentFlag;
    }

    @Override
    public String toString() {
        return issueDescription + " (Urgent: " + urgentFlag + ")";
    }
}

public class TicketQueueManager {
    public static void main(String[] args) {
        PriorityQueue<ServiceTicket> ticketQueue = new PriorityQueue<>(
            (t1, t2) -> Boolean.compare(t2.urgentFlag, t1.urgentFlag) // true before
false
        );

        ticketQueue.add(new ServiceTicket("Network Issue", false));
        ticketQueue.add(new ServiceTicket("Server Down", true));
        ticketQueue.add(new ServiceTicket("Password Reset", false));

        while (!ticketQueue.isEmpty()) {
```

```
            System.out.println("Next ticket: " + ticketQueue.poll());
        }
    }
}
```

Reasoning to interviewer

- A normal queue (LinkedList or ArrayDeque) would process in arrival order and can't prioritize urgent items.
- TreeSet could be used but doesn't handle duplicates well and requires unique tickets.
- PriorityQueue is specifically designed for priority-based retrieval, making it the cleanest solution here.
- The comparator determines priority logic; here we chose urgent first, but it can be extended (e.g., urgent + timestamp for FIFO among urgent).

Future scope

- For complex prioritization (e.g., urgent vs. SLA times), the comparator can consider multiple fields.
- For concurrency in multi-threaded support systems, wrap with PriorityBlockingQueue.
- Integrate with monitoring to automatically flag certain keywords (e.g., "server down") as urgent.

Scenario 28: Efficient min and max retrieval for a number set

We're recording numbers and frequently need to find the smallest and largest values. Sorting each time is too expensive, so we need a collection that maintains order dynamically.

Answer

A TreeSet<Integer> is ideal.

- It stores numbers in natural sorted order automatically.
- first() retrieves the smallest number and last() retrieves the largest in O(1).
- Insertions/removals are O(log n), which is efficient compared to repeatedly sorting a list.
- It also enforces uniqueness, preventing duplicate entries.

Code

```
import java.util.TreeSet;

public class NumberRecorder {
    public static void main(String[] args) {
        TreeSet<Integer> recordedNumbers = new TreeSet<>();

        recordedNumbers.add(5);
        recordedNumbers.add(10);
        recordedNumbers.add(3);
        recordedNumbers.add(10); // duplicate ignored

        System.out.println("Smallest number: " + recordedNumbers.first()); // 3
        System.out.println("Largest number: " + recordedNumbers.last());   // 10
    }
}
```

Reasoning to interviewer

- An ArrayList would require re-sorting or scanning for min/max each time (O(n)).
- A PriorityQueue can give either min or max efficiently, but not both at once without maintaining two heaps.
- TreeSet gives both min and max instantly while keeping the set sorted at all times.

- This makes it ideal for dynamic datasets where order-sensitive queries are frequent.

Future scope

- If duplicates are required, switch to TreeMap<Integer, Integer> (storing counts).
- For very large numeric datasets, consider specialized primitive collections (like fastutil) to avoid boxing overhead.
- For concurrent scenarios (e.g., shared analytics engine), ConcurrentSkipListSet is a thread-safe alternative.

Scenario 29: Auto-complete with alphabetical suggestions and frequency tracking

We need an auto-complete feature where:

- Suggestions are **unique**.
- They must appear in **alphabetical order**.
- Each suggestion also tracks its **frequency of occurrence**.

Answer

A TreeMap<String, Integer> is the best fit.

- Keys (suggestions) are stored in sorted order, so suggestions appear alphabetically by default.
- Values are integers representing how many times a suggestion was recorded.
- put() with getOrDefault() allows quick frequency updates in O(log n).
- The map enforces uniqueness because duplicate keys simply update the count instead of creating new entries.

Code

```
import java.util.TreeMap;

public class AutoCompleteFeature {
    public static void main(String[] args) {
        TreeMap<String, Integer> suggestionFrequency = new TreeMap<>();

        // Adding suggestions
        suggestionFrequency.put("Apple",
suggestionFrequency.getOrDefault("Apple", 0) + 1);
        suggestionFrequency.put("Banana",
suggestionFrequency.getOrDefault("Banana", 0) + 1);
        suggestionFrequency.put("Apple",
suggestionFrequency.getOrDefault("Apple", 0) + 1);

        System.out.println("Suggestions with frequencies: " + suggestionFrequency);
    }
}
```

Reasoning to interviewer

- A TreeSet would maintain alphabetical uniqueness but wouldn't track counts.

- A HashMap with counts could work, but order would be lost. We'd need sorting separately.
- TreeMap combines ordering and frequency tracking in one structure, making it ideal for an auto-complete system.
- With methods like subMap(prefix, nextPrefix), we can even fetch suggestions dynamically by prefix.

Future scope

- Extend values to hold richer metadata (e.g., last access time, click-through rate).
- Use NavigableMap methods like ceilingKey or floorKey for prefix-based navigation.
- For very large datasets, consider Trie structures for prefix efficiency while still keeping frequency counts.

Scenario 30: Dynamic leaderboard with fast top-scorer retrieval

We're building a competition leaderboard where scores are updated continuously, and we need:

- Efficient retrieval of the **top-scoring student** at any moment.
- Dynamic updates as new scores come in.

Answer

A PriorityQueue<Competitor> configured as a **max-heap** is ideal.

- By default, PriorityQueue is a min-heap, but with a custom comparator, we can make it prioritize highest scores first.
- The student with the top score is always at the head (peek()).
- Insertions and removals are O(log n), and access to the top element is O(1).

Code

```
import java.util.Comparator;
import java.util.PriorityQueue;

class Competitor {
    String fullName;
    int points;

    Competitor(String fullName, int points) {
        this.fullName = fullName;
        this.points = points;
    }

    @Override
    public String toString() {
        return fullName + ": " + points;
    }
}

public class Leaderboard {
    public static void main(String[] args) {
        PriorityQueue<Competitor> leaderboard = new PriorityQueue<>(
            Comparator.comparingInt((Competitor c) -> c.points).reversed()
        );

        leaderboard.add(new Competitor("Alice", 90));
        leaderboard.add(new Competitor("Bob", 95));
        leaderboard.add(new Competitor("Charlie", 85));
```

```
        System.out.println("Top scorer: " + leaderboard.peek());
    }
}
```

Reasoning to interviewer

- A TreeSet with a custom comparator could also maintain order, but updating scores (removing and re-adding) is less direct.
- A HashMap would allow fast lookup but wouldn't maintain a sorted ranking by scores.
- PriorityQueue ensures that the most important element (highest score) is always available in O(1).
- Perfect for real-time competitions where frequent updates happen and we only care about quick access to the current leader.

Future scope

- If multiple competitors can have the same score, extend comparator logic to break ties (e.g., earliest submission time).
- For showing a **full leaderboard**, a TreeMap<Integer, List<Competitor>> might complement the priority queue.
- For distributed competitions, shard leaderboards by region and merge top scores periodically.

Scenario 31: Tracking stock price updates with earliest and latest access

We're building a stock monitoring system where:

- Prices must be stored in the **order they arrive**.
- We need **quick access** to both the earliest and the most recent price.
- Frequent updates make shifting elements (like in ArrayList) too costly.

Answer

The best fit is an ArrayDeque<Double>.

- It preserves insertion order naturally.
- Provides O(1) operations for adding/removing at both ends.
- getFirst() and getLast() give constant-time access to the earliest and latest prices.
- Unlike ArrayList, it doesn't require shifting elements during removals at the front, making it more efficient for time-ordered data.

Code

```
import java.util.ArrayDeque;
import java.util.Deque;

public class PriceMonitor {
    public static void main(String[] args) {
        Deque<Double> priceUpdates = new ArrayDeque<>();

        priceUpdates.addLast(120.5);
        priceUpdates.addLast(121.8);
        priceUpdates.addLast(119.3);

        System.out.println("Earliest price: " + priceUpdates.getFirst()); // 120.5
        System.out.println("Latest price: " + priceUpdates.getLast());   // 119.3
    }
}
```

Reasoning to interviewer

- ArrayDeque is more memory-efficient than LinkedList and avoids per-node object overhead.

- ArrayList would require O(n) removals for oldest prices, which doesn’t scale well.
- Deque semantics (double-ended queue) align perfectly with stock price analysis, where both the head (earliest) and tail (latest) are critical.

Future scope

- If we need sliding windows (e.g., last N prices), we can simply remove oldest entries once size exceeds N.
- For multi-threaded monitoring, ConcurrentLinkedDeque is a safe alternative.
- If persistence is required, prices can be periodically flushed to a time-series database.

Scenario 32: LRU cache for a web application

We're building a cache that:

- Holds a fixed number of items.
- Automatically discards the **least recently used** entry when full.
- Should reorder items based on access history.

Answer

The right collection is a LinkedHashMap<K, V> with **access-order** enabled.

- By passing true for accessOrder in the constructor, the map reorders entries whenever they're accessed.
- Overriding removeEldestEntry() allows automatic eviction when size exceeds capacity.
- This is the classic way to implement an LRU cache in Java.

Code

```
import java.util.LinkedHashMap;
import java.util.Map;

class LRUCache<K, V> extends LinkedHashMap<K, V> {
    private final int capacity;

    public LRUCache(int capacity) {
        super(capacity, 0.75f, true); // true = access-order
        this.capacity = capacity;
    }

    @Override
    protected boolean removeEldestEntry(Map.Entry<K, V> eldest) {
        return size() > capacity;
    }
}

public class CacheDemo {
    public static void main(String[] args) {
        LRUCache<Integer, String> cache = new LRUCache<>(3);

        cache.put(1, "Alpha");
        cache.put(2, "Beta");
        cache.put(3, "Gamma");

        cache.get(1);   // Access item 1, moves it to recent position
```

```
        cache.put(4, "Delta"); // Cache exceeds size, evicts least recently used (key
2)

        System.out.println("Current Cache: " + cache);
        // Output: {3=Gamma, 1=Alpha, 4=Delta}
    }
}
```

Reasoning to interviewer

- A plain HashMap gives fast lookups but no ordering of usage.
- A TreeMap orders by key, not access history.
- LinkedHashMap in access-order mode is specifically designed for LRU-style behavior.
- With removeEldestEntry, eviction happens automatically—no manual intervention needed.

Future scope

- For production-grade caching with concurrency and eviction policies (TTL, size-based), libraries like **Caffeine** or **Guava Cache** are better.
- We can extend the cache to log evictions or sync evicted entries to a backing store.
- If we need thread-safety, wrap it with Collections.synchronizedMap() or use ConcurrentLinkedHashMap implementations.

Scenario 33: Storing user action logs with chronological order and fast ID lookup

We're building an analytics tool where:

- Logs must maintain the **order of actions** chronologically.
- Each log is uniquely identified by an **action ID**.
- We need fast retrieval of logs using these IDs.

Answer

A LinkedHashMap<Integer, String> is the best fit.

- LinkedHashMap preserves insertion order, so logs stay chronological.
- It provides O(1) average-time lookups by action ID, just like a HashMap.
- This makes it ideal for analytics tools that frequently need both ordered iteration and direct access by key.

Code

```
import java.util.LinkedHashMap;

public class UserActionLogger {
    public static void main(String[] args) {
        LinkedHashMap<Integer, String> actionLogs = new LinkedHashMap<>();

        actionLogs.put(101, "User Login");
        actionLogs.put(102, "Product Viewed");
        actionLogs.put(103, "Added to Cart");

        System.out.println("Action Logs: " + actionLogs);
        System.out.println("Action with ID 102: " + actionLogs.get(102));
    }
}
```

Reasoning to interviewer

- A HashMap alone would give fast lookups but wouldn't guarantee chronological order.
- A List would preserve order but make lookups by ID O(n).

- LinkedHashMap combines both requirements—ordered iteration and fast key access.
- For advanced queries (e.g., by timestamp ranges), pairing it with a TreeMap<Long, Action> could complement the solution.

Future scope

- Extend value type from String to a UserAction object containing timestamp, user, and metadata.
- Add a secondary index (like a TreeMap keyed by timestamp) for fast time-range analytics.
- For concurrency, wrap it with synchronization or use ConcurrentLinkedHashMap alternatives.

Scenario 34: Social media timeline with front insertions and back removals

We're building a social media timeline where:

- New posts are added to the **front** of the feed.
- Old posts are removed from the **back** when space is limited.
- Both operations must be efficient.

Answer

The best choice is a Deque<String>, specifically an ArrayDeque.

- addFirst() lets us push new posts to the front efficiently.
- removeLast() efficiently discards old posts from the back.
- Operations at both ends are O(1), making it scalable for timelines.

Code

```
import java.util.ArrayDeque;
import java.util.Deque;

public class Timeline {
    public static void main(String[] args) {
        Deque<String> posts = new ArrayDeque<>();

        posts.addFirst("First Post");
        posts.addFirst("Newest Post");
        posts.addFirst("Another New Post");

        System.out.println("Current Timeline: " + posts);

        // Removing the oldest post
        System.out.println("Removing oldest post: " + posts.removeLast());
    }
}
```

Reasoning to interviewer

- ArrayList would make front insertions and back removals O(n) due to shifting.
- LinkedList supports operations at both ends but has higher memory overhead.

- ArrayDeque is backed by a resizable array, giving low memory overhead and O(1) operations at both ends.
- Fits perfectly for feeds where freshness (front insertions) and pruning old data (back removals) are common.

Future scope

- Add a maximum capacity to automatically drop old posts once the feed grows beyond a limit.
- For multi-user feeds, use a Map<User, Deque<Post>> to maintain individual timelines.
- If persistence is needed, old posts can be flushed to a database before being removed.

Scenario 35: Chat system storing sequential messages (with duplicates allowed)

We're building a chat system where:

- Messages must be saved in the **exact order they were sent**.
- Duplicate messages (like repeated "Hi!") are valid and should be preserved.

Answer

The right choice is an ArrayList<String>.

- It preserves insertion order, so chat history is recorded exactly as messages arrive.
- It allows duplicates, which is essential in real conversations.
- It provides O(1) access by index, useful when displaying or paginating chat logs.

Code

```
import java.util.ArrayList;

public class ChatLog {
    public static void main(String[] args) {
        ArrayList<String> chatHistory = new ArrayList<>();

        chatHistory.add("Hi!");
        chatHistory.add("How are you?");
        chatHistory.add("Hi!"); // Duplicate allowed

        System.out.println("Chat messages: " + chatHistory);
    }
}
```

Reasoning to interviewer

- HashSet or LinkedHashSet would prevent duplicates, which doesn't fit this use case.
- TreeSet would sort messages alphabetically, losing the actual conversation order.
- ArrayList matches the real-world chat requirement: sequential order + duplicates preserved.

Future scope

- If message history grows too large, use a capped list or periodically flush to a database.
- For multi-user chatrooms, maintain a Map<ChatRoom, List<Message>>.
- If frequent deletions from the start are needed, a Deque could be considered instead of ArrayList.

Scenario 36: File tracker maintaining recent files (unique, ordered by access)

We're building a file tracking system where:

- Each file should appear only **once**.
- Order must reflect **most recent access**.
- If a file is accessed again, it should move to the latest position.

Answer

A LinkedHashSet<String> works well here.

- It ensures uniqueness, so files don't repeat.
- It preserves insertion order, so the list reflects access sequence.
- By removing and re-adding a file, we can update its position to the most recent access.

Code

```
import java.util.LinkedHashSet;

public class FileTracker {
    public static void main(String[] args) {
        LinkedHashSet<String> recentFiles = new LinkedHashSet<>();

        recentFiles.add("resume.pdf");
        recentFiles.add("photo.jpg");

        // Access resume.pdf again → remove & re-add to update order
        recentFiles.remove("resume.pdf");
        recentFiles.add("resume.pdf");

        System.out.println("Files accessed recently: " + recentFiles);
    }
}
```

Reasoning to interviewer

- A HashSet enforces uniqueness but doesn't maintain order.
- An ArrayList preserves order but allows duplicates.

- A LinkedHashSet balances both: uniqueness and order, making it ideal for “recently used” lists like editors or OS file pickers.
- The remove-and-readd trick ensures that re-accessed files float to the latest position.

Future scope

- If we want automatic eviction after reaching a max size, LinkedHashMap with access-order mode (true) is a cleaner LRU-style alternative.
- If persistence is needed, recent files can be synced with a config file or database.
- For multi-user systems, maintain a Map<User, LinkedHashSet<File>> to track per-user recent files.

Scenario 37: Multiplayer game queue with bidirectional processing

We need to maintain a queue of players where:

- Players join in the order they arrive.
- Sometimes we need to process from the **front** (standard queue behavior).
- Sometimes we need to process from the **back** (special cases like VIP processing or reverse order).

Answer

The right choice is a Deque<String>.

- Deque (double-ended queue) supports adding and removing elements from both ends.
- LinkedList implements Deque and provides flexibility for queue-style or stack-style operations.
- This fits perfectly for multiplayer systems where you sometimes dequeue in the usual order, but occasionally need to handle from the back.

Code

```
import java.util.Deque;
import java.util.LinkedList;

public class GameQueue {
    public static void main(String[] args) {
        Deque<String> playerQueue = new LinkedList<>();

        playerQueue.addLast("PlayerOne");  // normal join
        playerQueue.addLast("PlayerTwo");
        playerQueue.addFirst("VIPPlayer"); // VIP inserted at the front

        System.out.println("Next player to process: " + playerQueue.removeFirst()); //
VIP first
        System.out.println("Last player in line: " + playerQueue.removeLast());    //
PlayerTwo
    }
}
```

Reasoning to interviewer

- A simple Queue (like LinkedList or ArrayDeque) only guarantees operations at one end.
- A Stack would only allow LIFO processing, which doesn’t fit the queue requirements.
- Deque gives maximum flexibility: O(1) insertion and removal at both ends.
- Using LinkedList as the implementation also supports dynamic resizing without worrying about capacity.

Future scope

- For large-scale, concurrent multiplayer systems, ConcurrentLinkedDeque provides thread safety.
- If we need to enforce maximum queue size, we can use ArrayDeque with bounded capacity.
- Priority-based queues (e.g., PriorityBlockingQueue) can be layered if some players must skip the line based on ranking.

Scenario 38: Music playlist preserving order with duplicates allowed

We're building a music playlist feature where:

- Songs are stored in the **exact order** they are added.
- Duplicate tracks are allowed (e.g., same song appearing multiple times in the playlist).
- Indexed access is useful for playback controls like "play next" or "skip to track 5."

Answer

The right choice is an ArrayList<String>.

- It preserves insertion order, so songs play back in sequence.
- It allows duplicates, which is realistic for playlists.
- Indexed access is O(1), so jumping to a specific track number is efficient.

Code

```
import java.util.ArrayList;

public class MusicPlaylist {
    public static void main(String[] args) {
        ArrayList<String> playlist = new ArrayList<>();

        playlist.add("Track One");
        playlist.add("Track Two");
        playlist.add("Track One"); // duplicate allowed
        playlist.add("Track Three");

        System.out.println("Playlist songs: " + playlist);

        // Play second song
        System.out.println("Now playing: " + playlist.get(1));
    }
}
```

Reasoning to interviewer

- A HashSet or LinkedHashSet would prevent duplicates, which is not desirable here.
- A TreeSet would sort songs alphabetically, destroying the intended sequence.
- ArrayList gives exactly what we need: ordered sequence with duplicates intact, plus random access by index.

Future scope

- For shuffle or repeat modes, apply algorithms on top of the ArrayList.
- For large playlists where frequent deletions happen, consider LinkedList.
- If collaborative playlists are expected, use thread-safe variants like CopyOnWriteArrayList.
- Extend values to objects (Song with metadata) rather than just strings for richer functionality.

Scenario 39: Displaying recent search queries uniquely and in order

We're building a shopping app feature where:

- Search queries must appear in the **order they were made**.
- Each term should appear **only once**, even if repeated by the user.

Answer

The best fit is a LinkedHashSet<String>.

- It ensures uniqueness—duplicate queries are automatically ignored.
- It preserves insertion order, so the display matches the actual search sequence.
- Operations like add and contains run in O(1) average time, making it efficient for frequent updates.

Code

```
import java.util.LinkedHashSet;

public class SearchQueryHistory {
    public static void main(String[] args) {
        LinkedHashSet<String> recentQueries = new LinkedHashSet<>();

        recentQueries.add("Sneakers");
        recentQueries.add("Jackets");
        recentQueries.add("Sneakers"); // duplicate ignored
        recentQueries.add("Watches");

        System.out.println("Recent search terms: " + recentQueries);
    }
}
```

Reasoning to interviewer

- A HashSet prevents duplicates but doesn't preserve order.
- An ArrayList preserves order but allows duplicates unless explicitly filtered.
- LinkedHashSet combines both: queries are clean (no duplicates) and shown in the correct chronological order.
- This aligns perfectly with "recent search" UX in shopping apps.

Future scope

- To update the order when an old query is searched again, remove and re-add the term.
- For persistence, maintain the search history in a database but use LinkedHashSet in memory for fast access.
- If a maximum history size is required (say, last 10 queries), consider switching to LinkedHashMap with eviction logic.

Scenario 40: Task manager with priority-based execution

We're building a task manager where:

- Tasks have priorities.
- Higher-priority tasks must always be processed before lower-priority ones.

Answer

A PriorityQueue is the right choice.

- It organizes tasks either by natural order or with a custom comparator.
- The head of the queue always holds the task with the **highest priority** (lowest numeric value if natural order).
- Insertion and removal are O(log n), which is efficient for dynamic scheduling.

Code

```
import java.util.PriorityQueue;

public class PriorityTaskManager {
    public static void main(String[] args) {
        // Lower number = higher priority
        PriorityQueue<Integer> tasks = new PriorityQueue<>();

        tasks.add(3); // low priority
        tasks.add(1); // high priority
        tasks.add(2);

        while (!tasks.isEmpty()) {
            System.out.println("Processing task with priority: " + tasks.poll());
        }
    }
}
```

Reasoning to interviewer

- A normal queue (LinkedList, ArrayDeque) would process in arrival order, ignoring priority.
- A TreeSet could keep tasks sorted, but it disallows duplicates, which might be necessary for repeated tasks.

- PriorityQueue is purpose-built for scenarios where priority determines processing order.
- It’s widely used in schedulers, job execution engines, and simulations.

Future scope

- Replace Integer with a Task object containing fields like priority, deadline, and description, using a custom comparator.
- Use PriorityBlockingQueue for thread-safe concurrent task management.
- Combine with ExecutorService to implement real-world priority-based thread pools.

Scenario 41: Library system tracking borrowed books

You're designing a library system where books may be borrowed multiple times, and you want to keep track of all borrowings in the exact sequence.

Answer

An ArrayList<String> is the right fit here.

- It preserves insertion order, ensuring borrowings are recorded chronologically.
- It allows duplicates, so the same book can appear multiple times if borrowed repeatedly.
- Provides fast indexed access, useful for reports or replaying the borrow history.

Code

```
import java.util.ArrayList;

public class BorrowTracker {
    public static void main(String[] args) {
        ArrayList<String> borrowHistory = new ArrayList<>();

        borrowHistory.add("Harry Potter");
        borrowHistory.add("The Hobbit");
        borrowHistory.add("Harry Potter"); // borrowed again

        System.out.println("Borrow history: " + borrowHistory);
    }
}
```

Reasoning to interviewer

- A HashSet or TreeSet would remove duplicates, which isn't acceptable here.
- A LinkedHashSet would also filter out repeats, losing valuable borrow info.
- ArrayList aligns with the real-world requirement: chronological logging with duplicates.

Future scope

- Store richer details (book ID, borrower ID, timestamp) using a List<BookBorrowEvent> instead of strings.

- If frequent deletions from the start are required, switch to Deque.
- Persist borrow history in a database while keeping recent activity in memory.

Scenario 42: User management with unique email IDs

You're building a user management system where each user must be uniquely identified by their email ID.

Answer

The right choice is a HashSet<String> for uniqueness, or a HashMap<String, User> if we also need to store user details.

- HashSet ensures no duplicate emails.
- HashMap allows fast lookups and mapping email → user profile.
- Both give O(1) average-time operations, perfect for validating unique identifiers like emails.

Code

```
import java.util.HashMap;

class User {
    String email;
    String name;

    User(String email, String name) {
        this.email = email;
        this.name = name;
    }

    @Override
    public String toString() {
        return name + " (" + email + ")";
    }
}

public class UserDirectory {
    public static void main(String[] args) {
        HashMap<String, User> users = new HashMap<>();

        users.put("alice@example.com", new User("alice@example.com", "Alice"));
        users.put("bob@example.com", new User("bob@example.com", "Bob"));

        System.out.println("Lookup Alice: " + users.get("alice@example.com"));
    }
}
```

Reasoning to interviewer

- ArrayList would require linear searches to check uniqueness, inefficient for large datasets.
- TreeMap sorts keys, but sorting isn’t necessary for just enforcing uniqueness.
- HashMap (or HashSet) gives constant-time lookups and ensures uniqueness naturally.

Future scope

- For case-insensitive email handling, use a case-insensitive comparator or normalize before storage.
- In distributed systems, back this with a DB unique index on email.
- For concurrent access, use ConcurrentHashMap.

Scenario 43: Word counter for a document

You're implementing a word counter where words must be stored along with their frequencies.

Answer

A HashMap<String, Integer> is best here.

- Keys = words, Values = counts.
- It allows O(1) average-time updates per word.
- Perfect for text analytics, log parsing, or word frequency analysis.

Code

```
import java.util.HashMap;

public class WordCounter {
    public static void main(String[] args) {
        String text = "apple banana apple orange banana apple";
        String[] words = text.split(" ");

        HashMap<String, Integer> wordCount = new HashMap<>();

        for (String word : words) {
            wordCount.put(word, wordCount.getOrDefault(word, 0) + 1);
        }

        System.out.println("Word frequencies: " + wordCount);
    }
}
```

Reasoning to interviewer

- A List would require repeated scans to count, which is inefficient.
- A Set enforces uniqueness but can't track frequencies.
- HashMap naturally maps each word to its count and handles updates efficiently.

Future scope

- Use TreeMap if we want alphabetical ordering of words.

- Combine with streams for functional-style counting: Arrays.stream(words).collect(Collectors.groupingBy(w -> w, Collectors.counting()));
- For very large datasets, use parallel streams or Hadoop/Spark for distributed word counts.

Scenario 44: Ride-sharing app with indexed ride access

You are implementing a ride-sharing app where you need to maintain a list of ongoing rides and frequently access rides by index.

Answer

ArrayList<Ride> is the best fit.

- It preserves insertion order.
- Provides O(1) random access by index, which is essential if rides are displayed by position (e.g., in a dashboard).
- Insertion at the end is efficient, which matches the case of adding new rides.

Code

```
import java.util.ArrayList;

class Ride {
    int rideId;
    String driver;

    Ride(int rideId, String driver) {
        this.rideId = rideId;
        this.driver = driver;
    }

    @Override
    public String toString() {
        return "Ride{" + rideId + ", Driver=" + driver + "}";
    }
}

public class RideManager {
    public static void main(String[] args) {
        ArrayList<Ride> rides = new ArrayList<>();

        rides.add(new Ride(101, "Alice"));
        rides.add(new Ride(102, "Bob"));

        // Access by index
        System.out.println("Second ride: " + rides.get(1));
    }
}
```

Reasoning to interviewer

- LinkedList gives O(n) access for index lookups, not efficient for frequent random access.
- ArrayList has O(1) random access, making it ideal here.
- Since most operations are appending new rides and retrieving by index, ArrayList is optimal.

Future scope

- If rides need frequent removals from the middle, LinkedList could be considered.
- For concurrent ride management, use CopyOnWriteArrayList if reads dominate.
- Persist rides in a database but keep an in-memory list for active rides.

Scenario 45: Music player with frequent insertions/removals

In a music player app, songs are added and removed frequently from the playlist at arbitrary positions.

Answer

LinkedList<String> is a better fit.

- Insertions and deletions at arbitrary positions are O(1) once you have a reference to the node.
- Unlike ArrayList, it doesn't require shifting elements.
- Perfect for dynamic playlists where songs may be reordered or removed frequently.

Code

```
import java.util.LinkedList;

public class MusicPlayer {
    public static void main(String[] args) {
        LinkedList<String> playlist = new LinkedList<>();

        playlist.add("Song A");
        playlist.add("Song B");
        playlist.add("Song C");

        playlist.add(1, "New Song"); // Insert at position 1
        playlist.remove("Song C");   // Remove specific song

        System.out.println("Updated Playlist: " + playlist);
    }
}
```

Reasoning to interviewer

- ArrayList is costly for insertions/removals in the middle (O(n) due to shifting).
- LinkedList handles frequent structural modifications better.
- Tradeoff: random access is slower (O(n)), but that's acceptable for sequential playback.

Future scope

- If we want to support shuffle mode, maintain an auxiliary array for random access.

- For very large playlists, consider lazy loading songs instead of holding them all in memory.
- To allow navigation forward and backward, use ListIterator.

Scenario 46: Undo/redo feature with bi-directional navigation

You want to implement an undo/redo feature for a text editor where operations need to be traversed in both directions.

Answer

A LinkedList<String> with a ListIterator is ideal.

- ListIterator allows moving both forward (next()) and backward (previous()).
- Undo = move back, Redo = move forward.
- LinkedList ensures efficient additions/removals as user actions are stored.

Code

```
import java.util.LinkedList;
import java.util.ListIterator;

public class TextEditor {
    public static void main(String[] args) {
        LinkedList<String> actions = new LinkedList<>();

        actions.add("Typed A");
        actions.add("Typed B");
        actions.add("Deleted B");

        ListIterator<String> iterator = actions.listIterator(actions.size());

        // Undo
        if (iterator.hasPrevious()) {
            System.out.println("Undo: " + iterator.previous());
        }

        // Redo
        if (iterator.hasNext()) {
            System.out.println("Redo: " + iterator.next());
        }
    }
}
```

Reasoning to interviewer

- A simple Iterator only supports forward traversal, not enough for undo/redo.
- ArrayList would work but inserts/removals in the middle (when editing history) are expensive.

- LinkedList with ListIterator provides flexibility: bidirectional navigation + efficient history tracking.

Future scope

- Extend actions to store objects (Action class with metadata) instead of plain strings.
- Add limits to prevent unbounded memory growth (e.g., store last 100 actions).
- For multi-user collaborative editing, combine with version control mechanisms.

Scenario 47: Blog tag system with uniqueness, order doesn't matter

You're creating a tag system for blog posts where duplicate tags must be avoided, but display order isn't important.

Answer

Use a HashSet<String>.

- Enforces uniqueness automatically (no duplicate tags).
- Average O(1) adds/contains, great for frequent tag checks.
- Since order isn't required, we don't pay for maintaining one.

Code

```
import java.util.HashSet;

public class BlogTags {
    public static void main(String[] args) {
        HashSet<String> tags = new HashSet<>();

        tags.add("java");
        tags.add("collections");
        tags.add("java"); // duplicate ignored

        System.out.println("Unique tags: " + tags);
        System.out.println("Has 'java'? " + tags.contains("java"));
    }
}
```

Reasoning to interviewer

- ArrayList would allow duplicates and need O(n) checks to prevent them.
- LinkedHashSet and TreeSet maintain order; unnecessary overhead when order isn't needed.
- HashSet gives the best performance/space profile for this use case.

Future scope

- If you later want stable presentation order, switch to LinkedHashSet.
- For tag suggestions alphabetically, use TreeSet or sort a copy of the set on demand.
- For tag → posts mapping, pair with Map<String, Set<PostId>>.

Scenario 48: Recent searches—unique and shown in the exact order typed

You're building a recent search feature in an e-commerce app where searches must be unique and displayed in insertion order.

Answer

Use a LinkedHashSet<String>.

- Preserves insertion order for display.
- Prevents duplicates automatically.
- Average O(1) adds/lookups, perfect for a constantly changing "recent" list.

Code

```
import java.util.LinkedHashSet;

public class RecentSearches {
    public static void main(String[] args) {
        LinkedHashSet<String> recent = new LinkedHashSet<>();

        recent.add("laptop");
        recent.add("headphones");
        recent.add("laptop"); // duplicate ignored

        System.out.println("Recent searches: " + recent);

        // If user searches an old term again, move it to the end:
        String again = "headphones";
        if (recent.remove(again)) recent.add(again);

        System.out.println("After re-search: " + recent);
    }
}
```

Reasoning to interviewer

- HashSet would ensure uniqueness but lose the display order.
- ArrayList preserves order but needs manual duplicate checks/removals.
- LinkedHashSet is the sweet spot for "unique + order as typed," and the remove-then-add trick updates recency.

Future scope

- Cap the size (e.g., last 10) by dropping the eldest when exceeding capacity (consider a LinkedHashMap with eviction for that).
- Normalize queries (case-folding, trim) before storing to avoid near-duplicates.
- Persist to a small table per user for cross-device sync.

Scenario 49: Student ranking—unique scores in ascending order

You want a student ranking list where scores must be unique and always displayed from lowest to highest (or vice versa on demand).

Answer

Use a TreeSet<Integer> (or TreeSet<Score> if you'll store more than just the number).

- Maintains natural sorted order (ascending) automatically.
- Enforces uniqueness—duplicate scores won't be added.
- Supports range queries (headSet, tailSet, subSet) and quick min/max via first()/last().

Code

```
import java.util.TreeSet;

public class StudentRanking {
    public static void main(String[] args) {
        TreeSet<Integer> scores = new TreeSet<>();

        scores.add(78);
        scores.add(92);
        scores.add(85);
        scores.add(92); // duplicate ignored

        System.out.println("Scores ascending: " + scores);
        System.out.println("Top score: " + scores.last());
        System.out.println("Passing (>= 80): " + scores.tailSet(80));
    }
}
```

Reasoning to interviewer

- HashSet gives uniqueness but no ordering; you'd sort every time you display.
- LinkedHashSet preserves insertion order, not numeric rank.
- TreeSet keeps scores ordered continuously, so reads/range operations are cheap and clean.

Future scope

- If multiple students can share the same score, use TreeMap<Integer, java.util.List<Student>>.

- For descending display, iterate scores.descendingSet() or pass a reverse comparator at construction.
- If you need to attach student metadata and still sort by score, store a custom ScoreEntry and provide a comparator (and handle ties explicitly).

Scenario 50: Dictionary app with word-to-meaning mapping

You're developing a dictionary app where each word maps to its meaning, and fast lookups are required.

Answer

A HashMap<String, String> is the best choice.

- Provides O(1) average-time lookups by word.
- Prevents duplicate keys — each word is unique.
- Allows dynamic updates as new words are added.

Code

```
import java.util.HashMap;

public class DictionaryApp {
    public static void main(String[] args) {
        HashMap<String, String> dictionary = new HashMap<>();

        dictionary.put("Polymorphism", "The ability of an object to take on many
forms.");
        dictionary.put("Encapsulation", "Bundling data and methods into a single
unit.");

        System.out.println("Meaning of Polymorphism: " +
dictionary.get("Polymorphism"));
    }
}
```

Reasoning to interviewer

- A TreeMap would keep words sorted but adds overhead; not needed if alphabetical browsing isn't required.
- LinkedHashMap preserves insertion order, which doesn't matter in dictionary lookups.
- HashMap is the most efficient for direct word → meaning lookups.

Future scope

- Use TreeMap if alphabetical browsing is a feature.
- For synonyms, values could be List<String> instead of String.

- To support concurrency (like collaborative dictionary editing), switch to ConcurrentHashMap.

Scenario 51: Cache system with insertion order preserved

You're designing a cache where you must preserve the order in which entries were inserted, without duplicates.

Answer

A LinkedHashMap<K, V> is ideal.

- Maintains insertion order while mapping keys to values.
- Ensures uniqueness of keys.
- Provides O(1) average-time for get/put operations.

Code

```
import java.util.LinkedHashMap;

public class CacheSystem {
    public static void main(String[] args) {
        LinkedHashMap<Integer, String> cache = new LinkedHashMap<>();

        cache.put(1, "DataA");
        cache.put(2, "DataB");
        cache.put(3, "DataC");

        System.out.println("Cache contents in insertion order: " + cache);
    }
}
```

Reasoning to interviewer

- HashMap wouldn't preserve insertion order.
- TreeMap orders by keys, not by insertion sequence.
- LinkedHashMap gives predictable ordering — critical in cache implementations.

Future scope

- Enable **access-order** (true in constructor) to turn it into an LRU-style cache.
- Override removeEldestEntry to auto-evict old entries when capacity exceeds.
- For distributed caching, layer this over Redis or Memcached.

Scenario 52: Flight booking system sorted by departure time

You're implementing a flight booking system where flights must be listed in order of departure time.

Answer

A TreeMap<Long, String> (departure time as key) is the right choice.

- Keeps flights sorted by key (departure time).
- Provides efficient range queries (headMap, tailMap) for flights before/after a given time.
- Allows O(log n) insertions and lookups.

Code

```
import java.util.TreeMap;

public class FlightSchedule {
    public static void main(String[] args) {
        TreeMap<Long, String> flights = new TreeMap<>();

        flights.put(900L, "Flight A");
        flights.put(1100L, "Flight B");
        flights.put(1030L, "Flight C");

        System.out.println("All flights (sorted): " + flights);
        System.out.println("Flights before 11 AM: " + flights.headMap(1100L));
    }
}
```

Reasoning to interviewer

- HashMap gives fast lookups but no ordering, so sorting would be manual.
- LinkedHashMap preserves insertion order, but we need **time-based ordering**.
- TreeMap automatically sorts by departure time and supports range-based queries (perfect for schedules).

Future scope

- If multiple flights depart at the same time, map value should be a List<Flight>.
- For reverse chronological order (latest first), use descendingMap().
- For concurrency (real-time booking systems), use ConcurrentSkipListMap.

Scenario 53: Ticket booking — first-come, first-served processing

Customers should be served strictly in arrival order (FCFS). The queue grows and shrinks constantly as users book and cancel.

Answer

Use a Queue<T> backed by ArrayDeque<T>.

- ArrayDeque is faster and lighter than LinkedList for queue ops.
- Enqueue with add()/offer(), dequeue with poll(); both are O(1) amortized.
- No need for random access or sorting—just strict FIFO.

Code

```
import java.util.ArrayDeque;
import java.util.Queue;

public class TicketCounter {
    public static void main(String[] args) {
        Queue<String> queue = new ArrayDeque<>();

        queue.offer("UserA");
        queue.offer("UserB");
        queue.offer("UserC");

        System.out.println("Serving: " + queue.poll()); // UserA
        System.out.println("Next up: " + queue.peek()); // UserB
    }
}
```

Reasoning to interviewer

- ArrayDeque outperforms LinkedList for pure queue behavior (less GC pressure, better cache locality).
- A PriorityQueue would change ordering semantics (priority over arrival)—not desired for FCFS.
- Simplicity matters: FIFO semantics map cleanly to Queue APIs.

Future scope

- For multi-threaded producers/consumers, use ConcurrentLinkedQueue or LinkedBlockingQueue.
- Add admission control (max queue size) via offer + checks.

- Emit metrics (queue length, wait time) to detect overload.

Scenario 54: Task scheduler — higher priority first

Background jobs (e.g., report generation, thumbnailing) must run by priority; ties should respect earlier submission time.

Answer

Use a PriorityQueue<Task> with a comparator: higher priority first, tie-break on enqueuedAt.

- Head of the queue is always the highest priority task.
- Insert/remove are O(log n); peek is O(1).
- Deterministic tie-breaking avoids starvation and flapping.

Code

```
import java.util.PriorityQueue;
import java.util.Comparator;

class Task {
    final int priority;        // higher number = more important
    final long enqueuedAt;     // millis
    final String description;

    Task(int priority, long enqueuedAt, String description) {
        this.priority = priority;
        this.enqueuedAt = enqueuedAt;
        this.description = description;
    }
    @Override public String toString() { return description + " [p=" + priority + "]"; }
}

public class TaskScheduler {
    public static void main(String[] args) {
        PriorityQueue<Task> pq = new PriorityQueue<>(
            Comparator.<Task>comparingInt(t -> t.priority).reversed()
                    .thenComparingLong(t -> t.enqueuedAt)
        );

        pq.add(new Task(1, System.currentTimeMillis(), "Low: sync metrics"));
        pq.add(new Task(3, System.currentTimeMillis(), "High: generate report"));
        pq.add(new Task(3, System.currentTimeMillis() + 1, "High: export CSV"));

        System.out.println("Run first: " + pq.poll()); // High: generate report
    }
```

```
}
```

Reasoning to interviewer

- Queue/Deque enforce arrival order, not business priority.
- TreeSet can sort but forbids equal keys unless you encode uniqueness; PriorityQueue naturally allows duplicates.
- Comparator captures evolving business rules (e.g., SLA deadlines, aging boosts).

Future scope

- For concurrent producers/consumers, use PriorityBlockingQueue.
- Add aging: increase effective priority the longer a task waits.
- Track throughput/latency; autoscale workers based on queue depth.

Scenario 55: Messaging buffer — add/remove from both ends

A chat/notification pipeline needs to push new messages, pop newest for moderation, or trim oldest when the buffer exceeds a size.

Answer

Use a Deque<String> implemented by ArrayDeque.

- O(1) adds/removes at both ends: addFirst/addLast, removeFirst/removeLast.
- Maintains arrival order but gives flexibility to operate from either end.
- More memory-efficient than LinkedList (no node objects).

Code

```
import java.util.ArrayDeque;
import java.util.Deque;

public class MessageBuffer {
    public static void main(String[] args) {
        Deque<String> buffer = new ArrayDeque<>();

        buffer.addLast("msg-1");       // normal append
        buffer.addLast("msg-2");
        buffer.addFirst("admin-pinned"); // urgent to front

        System.out.println("Process oldest: " + buffer.removeFirst()); // admin-pinned
        System.out.println("Drop newest: " + buffer.removeLast());     // msg-2
        System.out.println("Remaining head: " + buffer.peekFirst());   // msg-1
    }
}
```

Reasoning to interviewer

- ArrayList makes front removals O(n) due to shifting.
- LinkedList supports deque ops but with higher per-element overhead.
- ArrayDeque is purpose-built for double-ended queues with tight performance.

Future scope

- Add a max capacity; before addLast, if size()==MAX, call removeFirst() to behave like a ring buffer.
- For multi-threaded pipelines, switch to ConcurrentLinkedDeque or LinkedBlockingDeque.

- Attach timestamps and prune by age (e.g., drop messages older than N minutes).

Scenario 56: Bi-directional contact browsing

You're building a contacts screen where users can swipe forward and backward through contacts. You also need to insert or delete contacts at the current cursor position smoothly.

Answer

Use a LinkedList<Contact> with a ListIterator<Contact>.

- ListIterator supports next() and previous() for two-way navigation.
- It also supports add(), remove(), and set() at the iterator's current position without index gymnastics.
- LinkedList keeps insertions/removals O(1) once the iterator is positioned.

Code

```
import java.util.LinkedList;
import java.util.ListIterator;

class Contact {
    final String name;
    Contact(String name) { this.name = name; }
    @Override public String toString() { return name; }
}

public class ContactBrowser {
    public static void main(String[] args) {
        LinkedList<Contact> contacts = new LinkedList<>();
        contacts.add(new Contact("Ava"));
        contacts.add(new Contact("Ben"));
        contacts.add(new Contact("Chloe"));

        ListIterator<Contact> it = contacts.listIterator(); // at start

        System.out.println("Forward: " + it.next());   // Ava
        it.add(new Contact("Ava (Work)"));            // insert after Ava
        System.out.println("Forward: " + it.next());   // Ben
        System.out.println("Backward: " + it.previous()); // Ben
        it.remove();                                  // remove Ben

        System.out.println("All contacts: " + contacts);
    }
}
```

Reasoning to interviewer

- A plain Iterator only moves forward and lacks add/set.

- With ListIterator, edits occur at the iterator position safely and cheaply.
- ArrayList would make mid-list insert/delete O(n) due to shifting; LinkedList avoids that overhead.

Future scope

- Keep the iterator as UI cursor; map swipe gestures to next/previous.
- For sorted views, use a separate sorted list or keep a comparator and rebuild as needed.
- If reads dominate and the list is large and stable, consider ArrayList plus indices; but switch back to LinkedList when edits are frequent.

Scenario 57: Safely removing elements while iterating

You need to scan a list of orders and delete those that are cancelled. Doing for (Order o : list) { if(cancelled) list.remove(o); } risks ConcurrentModificationException.

Answer

Iterate with an Iterator<Order> and call iterator.remove() when you decide to delete.

- iterator.remove() is the only safe in-place removal during iteration for standard lists.
- Avoid removing via the list API inside the loop.

Code

```
import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;

class Order {
    final int id; final boolean cancelled;
    Order(int id, boolean cancelled) { this.id = id; this.cancelled = cancelled; }
    @Override public String toString() { return "Order#" + id; }
}

public class OrderCleanup {
    public static void main(String[] args) {
        List<Order> orders = new ArrayList<>();
        orders.add(new Order(1, false));
        orders.add(new Order(2, true));
        orders.add(new Order(3, true));

        for (Iterator<Order> it = orders.iterator(); it.hasNext(); ) {
            Order o = it.next();
            if (o.cancelled) it.remove(); // safe removal
        }

        System.out.println("Remaining: " + orders);
    }
}
```

Reasoning to interviewer

- Structural modifications through the collection while a fail-fast iterator is active trigger ConcurrentModificationException.
- The iterator tracks expected mod count; its remove() updates it consistently.
- This pattern keeps time complexity linear and avoids defensive copies.

Future scope

- For bulk predicates, Java 8’s removeIf(predicate) is concise and optimized.
- In concurrent contexts, prefer concurrent collections or snapshot iteration (e.g., CopyOnWriteArrayList for read-heavy cases).
- Log removed items if you need an audit trail.

Scenario 58: Updating elements during traversal

You're processing a list of student records to curve grades in place (e.g., +5 points) and sometimes insert a "Make-Up Exam" record right after a failing grade.

Answer

Use a ListIterator<Student> so you can both update current elements with set() and insert new ones with add() during iteration.

- ListIterator.set() replaces the last returned element.
- ListIterator.add() inserts at the iterator's cursor without breaking iteration.

Code

```
import java.util.ArrayList;
import java.util.List;
import java.util.ListIterator;

class Student {
    final String name; int grade;
    Student(String name, int grade) { this.name = name; this.grade = grade; }
    @Override public String toString() { return name + ":" + grade; }
}

public class GradeCurver {
    public static void main(String[] args) {
        List<Student> roster = new ArrayList<>();
        roster.add(new Student("Ava", 78));
        roster.add(new Student("Ben", 49));
        roster.add(new Student("Chloe", 88));

        for (ListIterator<Student> it = roster.listIterator(); it.hasNext(); ) {
            Student s = it.next();
            s.grade = Math.min(100, s.grade + 5); // curve in place
            it.set(s); // explicit, though mutating the object is enough for reference types

            if (s.grade < 50) {
                it.add(new Student(s.name + " (Make-Up)", 0)); // insert after failing
student
            }
        }

        System.out.println("Adjusted roster: " + roster);
    }
}
```

Reasoning to interviewer

- A plain Iterator can’t set() or add(); ListIterator is designed for in-place updates and local insertions.
- Doing list.add(index, ...) while iterating by index is error-prone and O(n) for ArrayList shifts; the iterator keeps cursor consistency.
- Complexity stays linear with localized operations.

Future scope

- If many inserts happen in the middle and the list is huge, consider LinkedList to avoid shifting.
- For transforms without structural changes, a stream + map to a new list is often cleaner.
- Add validation (cap grades at 100, track changes for audit) and collect stats while iterating.

Scenario 59: Default sorting of employees by numeric ID

You're building an employee directory where employees must be sorted by their ID numbers by default (natural order).

Answer

Implement Comparable<Employee> on the entity and define compareTo using the ID.

- Makes the "natural" sort of List<Employee> or TreeSet<Employee> align with business needs.
- Keeps the default consistent anywhere sorting is invoked (e.g., Collections.sort, Stream.sorted()).

Code

```
import java.util.*;

class Employee implements Comparable<Employee> {
    private final int id;
    private final String name;

    Employee(int id, String name) { this.id = id; this.name = name; }
    public int getId() { return id; }
    public String getName() { return name; }

    @Override
    public int compareTo(Employee other) {
        return Integer.compare(this.id, other.id);
    }

    @Override public String toString() { return id + " - " + name; }
}

public class Directory {
    public static void main(String[] args) {
        List<Employee> emps = Arrays.asList(
            new Employee(102, "Ava"),
            new Employee(101, "Ben"),
            new Employee(110, "Chloe")
        );

        Collections.sort(emps); // uses Comparable
        System.out.println(emps);
    }
}
```

Reasoning to interviewer

- Comparable defines a single, canonical ordering; perfect when one natural order exists (ID).
- Using ID avoids ambiguity and is stable across the app.
- If a TreeSet<Employee> is used, this ordering prevents inconsistent “duplicate” detection (tree equality is order-based).

Future scope

- If multiple natural orders are possible, keep Comparable as ID and expose alternates via Comparators (name, join date).
- Ensure compareTo is consistent with equals if the type will be used in sorted sets/maps.
- For large collections with frequent sorts, consider keeping them sorted in a TreeSet/TreeMap.

Scenario 60: Sort products by price or by name based on user choice

In a shopping cart, users can toggle sort: by price (ascending) or by name (A→Z).

Answer

Keep the model class plain (no Comparable), and provide two Comparators: by price and by name.

- This gives flexible, strategy-based sorting at runtime without baking a single "natural" order into the type.

Code

```
import java.util.*;
import static java.util.Comparator.*;

class Product {
    final String name;
    final double price;

    Product(String name, double price) { this.name = name; this.price = price; }
    @Override public String toString() { return name + " ($" + price + ")"; }
}

public class CartSorting {
    public static void main(String[] args) {
        List<Product> items = new ArrayList<>(Arrays.asList(
            new Product("Mouse", 19.99),
            new Product("Keyboard", 49.99),
            new Product("Monitor", 129.99)
        ));

        Comparator<Product> byPrice = comparingDouble(p -> p.price);
        Comparator<Product> byName  = comparing(p -> p.name,
String.CASE_INSENSITIVE_ORDER);

        // user chooses "price"
        items.sort(byPrice);
        System.out.println("By price: " + items);

        // user switches to "name"
        items.sort(byName);
        System.out.println("By name: " + items);
    }
}
```

Reasoning to interviewer

- Comparator is ideal when you need multiple orderings; you compose them on demand.
- Keeps domain class simple and avoids forcing one default order that might not always apply.
- With Java 8+, Comparator.comparing and method references make it concise and readable.

Future scope

- Add secondary keys: byPrice.thenComparing(byName) for deterministic ordering.
- Expose sort as an enum → comparator map (e.g., Map<SortMode, Comparator<Product>>).
- For internationalization, use a Collator for locale-sensitive name sorting.

Scenario 61: Movies sorted by release year, then title as a tiebreaker

You're designing a movie database: primary sort by release year (oldest→newest), but when years match, sort alphabetically by title.

Answer

Use a Comparator<Movie> that chains comparisons with thenComparing.

- Cleanly expresses multi-key ordering without embedding it in the entity's natural order.

Code

```
import java.util.*;
import static java.util.Comparator.*;

class Movie {
    final String title;
    final int year;

    Movie(String title, int year) { this.title = title; this.year = year; }
    @Override public String toString() { return year + " - " + title; }
}

public class MovieSort {
    public static void main(String[] args) {
        List<Movie> movies = Arrays.asList(
            new Movie("Dune", 2021),
            new Movie("Arrival", 2016),
            new Movie("Blade Runner 2049", 2017),
            new Movie("Another Dune", 2021)
        );

        Comparator<Movie> byYearThenTitle =
            comparingInt((Movie m) -> m.year)
            .thenComparing(m -> m.title, String.CASE_INSENSITIVE_ORDER);

        movies.sort(byYearThenTitle);
        System.out.println(movies);
    }
}
```

Reasoning to interviewer

- Chained comparators express domain rules directly: primary key (year), secondary key (title).
- Deterministic ordering avoids flicker in UIs and makes pagination stable.

- Using CASE_INSENSITIVE_ORDER prevents “Zebra” < “apple” surprises due to case.

Future scope

- Support descending year with comparingInt(Movie::getYear).reversed() if needed.
- Add tertiary keys (e.g., rating) with .thenComparingInt(Movie::getRating).
- For large datasets from DB, push sorting into SQL but keep the comparator for in-memory merges.

Scenario 62: Sorting a leaderboard in descending order

You're building a leaderboard feature where scores must be displayed from highest to lowest before showing them to users.

Answer

Use Collections.sort(list, comparator) with Collections.reverseOrder() or a custom comparator.

- Default sort is ascending; we flip it for descending.
- This avoids manually reversing after sorting.

Code

```
import java.util.*;

public class LeaderboardSorter {
    public static void main(String[] args) {
        List<Integer> scores = new ArrayList<>(Arrays.asList(50, 120, 90, 75));

        // Sort descending
        Collections.sort(scores, Collections.reverseOrder());

        System.out.println("Leaderboard: " + scores);
    }
}
```

Reasoning to interviewer

- Collections.sort is a utility wrapper around List.sort for pre-Java 8; still widely used.
- Reverse order gives a clean O(n log n) solution without extra passes.
- Could also use scores.sort(Comparator.reverseOrder()) in Java 8+.

Future scope

- Use multi-field sorting (e.g., score desc, then name asc) with custom comparators.
- For huge leaderboards, offload to DB sorting and only use Collections API for final adjustments.
- For stable ordering in ties, use thenComparing.

Scenario 63: Binary search for quick existence check

You have a sorted list of employee names and need to check if "Alice" exists efficiently.

Answer

Use Collections.binarySearch(list, key) on the sorted list.

- Binary search runs in O(log n), much faster than linear scan.
- Works only if the list is already sorted in natural or comparator-defined order.

Code

```
import java.util.*;

public class EmployeeSearch {
    public static void main(String[] args) {
        List<String> employees = new ArrayList<>(Arrays.asList("Alice", "Bob",
"Charlie", "David"));
        Collections.sort(employees); // must be sorted

        int index = Collections.binarySearch(employees, "Alice");

        if (index >= 0) {
            System.out.println("Found Alice at index " + index);
        } else {
            System.out.println("Alice not found.");
        }
    }
}
```

Reasoning to interviewer

- contains() on a list is O(n); binarySearch is O(log n).
- Works only on sorted data; otherwise results are undefined.
- If frequent lookups are required, a Set or Map may be more appropriate — but binary search is optimal when the list must stay sorted anyway.

Future scope

- For custom ordering (e.g., case-insensitive), sort with comparator and pass same comparator to binarySearch.
- If lookups dominate, switch to HashSet instead of List.

- For very large lists, consider external search structures (B-trees, DB indexes).

Scenario 64: Shuffling a playlist randomly

You want to shuffle songs in a playlist so playback order is randomized each time the user starts.

Answer

Use Collections.shuffle(list).

- Randomizes list order in O(n).
- Provides uniform randomness using Fisher–Yates algorithm.
- Perfect for “shuffle mode” in playlists.

Code

```
import java.util.*;

public class PlaylistShuffle {
    public static void main(String[] args) {
        List<String> playlist = new ArrayList<>(Arrays.asList(
            "Song A", "Song B", "Song C", "Song D"
        ));

        Collections.shuffle(playlist);

        System.out.println("Shuffled playlist: " + playlist);
    }
}
```

Reasoning to interviewer

- A naive Math.random() swap implementation is error-prone; Collections.shuffle is tested and unbiased.
- If you need reproducible randomness (e.g., for testing), pass a Random instance with a fixed seed.
- Shuffling is O(n), which is efficient even for large playlists.

Future scope

- Add a “repeat all” mode by looping shuffled playlist.
- Shuffle only a sublist (Collections.shuffle(list.subList(...))).
- Use Streams API (Collections.shuffle internally) to pipeline with filters before playback.

Scenario 65: Counting high-value transactions with Streams

You're analyzing sales data and want to count how many transactions are above **$1000**.

Answer

Use stream().filter(...).count().

- Streams make it concise and expressive.
- Filtering + counting avoids manual loops.

Code

```
import java.util.*;

public class SalesAnalyzer {
    public static void main(String[] args) {
        List<Integer> transactions = Arrays.asList(500, 1200, 800, 2500, 300);

        long highValueCount = transactions.stream()
                                          .filter(amount -> amount > 1000)
                                          .count();

        System.out.println("Transactions > $1000: " + highValueCount);
    }
}
```

Reasoning to interviewer

- A loop with a counter works, but Streams express the intent clearly.
- Filtering is lazy and efficient; count() is a terminal operation.
- Perfect for analytics dashboards with clear logic.

Future scope

- Use parallelStream() for huge datasets (though measure before).
- Switch to Collectors.partitioningBy(amount -> amount > 1000) for grouped counts.
- Extend to compute sums or averages using mapToInt().sum() / average().

Scenario 66: Getting top 5 highest-rated movies

You're working on a movie app and need to fetch the **top 5 movies** by rating.

Answer

Use stream().sorted(...).limit(5).toList().

- Sort by rating in descending order.
- Limit ensures only top 5 are retrieved.

Code

```
import java.util.*;
import static java.util.Comparator.*;

class Movie {
    final String title;
    final double rating;
    Movie(String title, double rating) { this.title = title; this.rating = rating; }
    @Override public String toString() { return title + " (" + rating + ")"; }
}

public class MovieApp {
    public static void main(String[] args) {
        List<Movie> movies = Arrays.asList(
            new Movie("Inception", 8.8),
            new Movie("Interstellar", 8.6),
            new Movie("Dunkirk", 7.9),
            new Movie("Tenet", 7.3),
            new Movie("Memento", 8.4),
            new Movie("The Prestige", 8.5)
        );

        List<Movie> top5 = movies.stream()
                                 .sorted(comparingDouble((Movie m) -> m.rating).reversed())
                                 .limit(5)
                                 .toList();

        System.out.println("Top 5 movies: " + top5);
    }
}
```

Reasoning to interviewer

- A manual sort + sublist works, but Streams make it expressive and concise.
- Sorting by comparator ensures highest-rated first.
- Easy to adjust (e.g., top 10, filter by genre first).

Future scope

- Add .filter(m -> m.rating >= 8) before sorting for threshold-based lists.
- Use Collectors.groupingBy for category-based leaderboards.
- For performance, consider priority queues if you only need top-k, not full sort.

Scenario 67: Removing empty strings from search queries

You have a list of search queries and want to drop all **empty or blank strings**.

Answer

Use stream().filter(s -> !s.trim().isEmpty()).toList().

- Filters out empties in a clean, functional style.
- No need for manual iteration.

Code

```
import java.util.*;

public class QueryCleaner {
    public static void main(String[] args) {
        List<String> queries = Arrays.asList("Shoes", "", "  ", "Bags", "Laptop");

        List<String> cleaned = queries.stream()
                                .filter(s -> !s.trim().isEmpty())
                                .toList();

        System.out.println("Cleaned queries: " + cleaned);
    }
}
```

Reasoning to interviewer

- A loop with if-check works but is verbose.
- Stream pipeline makes intent explicit: filter out empties.
- Handles both "" and " " after trimming.

Future scope

- Use .map(String::toLowerCase) to normalize queries.
- Combine with .distinct() to remove duplicates in the same pipeline.
- Persist cleaned queries back to DB or analytics.

Scenario 68: High-concurrency session store with fast reads/writes

You're building an API gateway that tracks active sessions by token. Multiple threads (request handlers) read and update sessions concurrently, and lock contention must be minimal.

Answer

Use ConcurrentHashMap<String, Session> for a thread-safe, high-throughput key→value store.

- Lock striping and non-blocking reads give great scalability.
- Supports atomic operations like putIfAbsent, compute, and computeIfAbsent to avoid race conditions.
- No global lock like Collections.synchronizedMap.

Code

```
import java.util.concurrent.*;

class Session {
    final String token;
    volatile long lastSeen;
    Session(String token, long lastSeen) { this.token = token; this.lastSeen =
lastSeen; }
    @Override public String toString() { return token + "@" + lastSeen; }
}

public class SessionStore {
    private final ConcurrentHashMap<String, Session> sessions = new
ConcurrentHashMap<>();

    public Session getOrCreate(String token) {
        return sessions.computeIfAbsent(token, t -> new Session(t,
System.currentTimeMillis()));
    }

    public void touch(String token) {
        sessions.computeIfPresent(token, (t, s) -> { s.lastSeen =
System.currentTimeMillis(); return s; });
    }

    public void removeIfExpired(long cutoff) {
        sessions.forEach((t, s) -> { if (s.lastSeen < cutoff) sessions.remove(t, s); }); //
weakly consistent
    }

    public static void main(String[] args) {
```

```
        SessionStore store = new SessionStore();
        Session s1 = store.getOrCreate("abc");
        store.touch("abc");
        store.removeIfExpired(System.currentTimeMillis() - 1000);
        System.out.println("Active: " + s1);
    }
}
```

Reasoning to interviewer

- HashMap is unsafe under concurrency; synchronizedMap serializes access.
- ConcurrentHashMap allows high read concurrency and fine-grained locking on writes.
- Atomic combinators (compute*) eliminate “check-then-act” races.

Future scope

- If ordering by key or range queries are needed, consider ConcurrentSkipListMap.
- For eviction/TTL, pair with a scheduler or use a caching library (Caffeine) that integrates with CHM.
- Export metrics (size, hit/miss) for capacity planning.

Scenario 69: Read-heavy subscribers list with infrequent writes

You maintain a list of websocket subscribers. Reads (broadcasts/iteration) are frequent; additions/removals are rare. Iteration must be thread-safe without external locks.

Answer

Use CopyOnWriteArrayList<Subscriber>.

- Iteration happens over a stable snapshot with no locking.
- Adds/removes create a new copy—expensive but acceptable when writes are infrequent.
- Ideal for event listeners, observers, and subscription lists.

Code

```
import java.util.concurrent.CopyOnWriteArrayList;

interface Subscriber { void onEvent(String msg); }

public class Broadcaster {
    private final CopyOnWriteArrayList<Subscriber> subs = new
CopyOnWriteArrayList<>();

    public void subscribe(Subscriber s) { subs.addIfAbsent(s); }
    public void unsubscribe(Subscriber s) { subs.remove(s); }

    public void broadcast(String msg) {
        for (Subscriber s : subs) { // lock-free snapshot iteration
            s.onEvent(msg);
        }
    }

    public static void main(String[] args) {
        Broadcaster b = new Broadcaster();
        b.subscribe(m -> System.out.println("A: " + m));
        b.subscribe(m -> System.out.println("B: " + m));
        b.broadcast("Hello");
    }
}
```

Reasoning to interviewer

- Collections.synchronizedList requires external synchronization during iteration to avoid CME.

- CopyOnWriteArrayList trades write cost for lock-free, CME-free reads—perfect for read-dominant workloads.
- Snapshot semantics mean a broadcast sees a consistent subscriber set.

Future scope

- If writes become frequent, switch to ConcurrentLinkedQueue + copy snapshot when broadcasting.
- For ordering guarantees or priority delivery, keep parallel structures (e.g., by topic).
- Add backpressure or batching for large subscriber counts.

Scenario 70: Producer–consumer pipeline with backpressure

You ingest jobs from HTTP and process them with worker threads. You need a bounded queue to apply backpressure when producers outrun consumers.

Answer

Use a BlockingQueue<Job> such as LinkedBlockingQueue<>(capacity).

- Producers call put()/offer(); consumers call take()/poll().
- If the queue is full, producers block (or fail fast with timeout), giving natural backpressure.
- Thread-safe; ideal for decoupling ingestion from processing.

Code

```
import java.util.concurrent.*;

class Job { final String payload; Job(String p) { this.payload = p; } }

public class Pipeline {
    private final BlockingQueue<Job> queue = new LinkedBlockingQueue<>(100);

    public void produce(String payload) throws InterruptedException {
        queue.put(new Job(payload)); // blocks if full
    }

    public void startConsumers(int n) {
        ExecutorService pool = Executors.newFixedThreadPool(n);
        for (int i = 0; i < n; i++) {
            pool.submit(() -> {
                try {
                    while (!Thread.currentThread().isInterrupted()) {
                        Job j = queue.take(); // blocks if empty
                        process(j);
                    }
                } catch (InterruptedException ie) {
                    Thread.currentThread().interrupt();
                }
            });
        }
    }

    private void process(Job j) {
        // do work
        System.out.println("Processed: " + j.payload);
    }
```

```
    public static void main(String[] args) throws Exception {
        Pipeline p = new Pipeline();
        p.startConsumers(2);
        for (int i = 0; i < 5; i++) p.produce("job-" + i);
    }
}
```

Reasoning to interviewer

- ArrayDeque/LinkedList aren’t thread-safe; manual locking adds complexity.
- BlockingQueue provides built-in coordination and backpressure, avoiding busy-waiting.
- Bounded capacity protects memory and stabilizes throughput under load.

Future scope

- Use PriorityBlockingQueue when jobs have priorities.
- Use DelayQueue for scheduled execution (run after a delay/at time).
- Add metrics, retries, and poison pills for graceful shutdown.

Scenario 71: Concurrent priority scheduling (highest priority first)

You're building a background job runner where jobs have priorities. Multiple producer threads enqueue jobs; multiple consumer threads must always take the highest-priority job available.

Answer

Use a PriorityBlockingQueue<Job> with a comparator that orders jobs by descending priority.

- Thread-safe, unbounded by default (you can wrap to bound it).
- Consumers block on take(); the head is always the highest-priority job.
- Allows multiple producers/consumers without extra locks.

Code

```
import java.util.Comparator;
import java.util.concurrent.*;

class Job {
    final int priority;  // higher = more important
    final String id;
    Job(int priority, String id) { this.priority = priority; this.id = id; }
    @Override public String toString() { return id + " [p=" + priority + "]"; }
}

public class PriorityWorkerPool {
    private final PriorityBlockingQueue<Job> queue =
        new PriorityBlockingQueue<>(11, Comparator.comparingInt((Job j) ->
j.priority).reversed());

    public void submit(Job j) { queue.put(j); } // never blocks (unbounded)
    public void startConsumers(int n) {
        ExecutorService pool = Executors.newFixedThreadPool(n);
        for (int i = 0; i < n; i++) {
            pool.submit(() -> {
                try {
                    while (!Thread.currentThread().isInterrupted()) {
                        Job j = queue.take(); // blocks if empty
                        System.out.println("Processing " + j);
                        // do work...
                    }
                } catch (InterruptedException ie) {
                    Thread.currentThread().interrupt();
                }
            });
        }
    }
```

```
    public static void main(String[] args) {
        PriorityWorkerPool w = new PriorityWorkerPool();
        w.startConsumers(2);
        w.submit(new Job(1, "low-1"));
        w.submit(new Job(3, "high-1"));
        w.submit(new Job(2, "med-1"));
    }
}
```

Reasoning to interviewer

- BlockingQueue variants handle coordination; PriorityBlockingQueue adds priority ordering.
- Unlike PriorityQueue, this one is thread-safe and supports blocking take().
- Perfect when correctness requires “highest-priority next” across many threads.

Future scope

- If you require bounded memory, wrap submissions with a semaphore or use a custom handoff.
- Add aging logic (increase effective priority for long-waiting jobs).
- For fairness across tenants, maintain one PBQ per tenant and merge with a dispatcher.

Scenario 72: Delayed execution / scheduled jobs

You need to schedule tasks to run after a delay (e.g., email reminders in 15 minutes, retries after backoff). Multiple threads will poll the next due task.

Answer

Use a DelayQueue<DelayedTask> where each element implements Delayed and exposes its scheduled time.

- Consumers block on take() until the head task's delay has expired.
- Ideal for retry queues, backoff, and "run-after" scheduling without timers per task.

Code

```
import java.util.concurrent.*;
import java.util.concurrent.atomic.AtomicLong;

class DelayedTask implements Delayed {
    private static final AtomicLong SEQ = new AtomicLong(0);
    private final long runAtNanos;
    private final long seq; // tie-breaker
    final String id;

    DelayedTask(String id, long delayMillis) {
        this.id = id;
        this.runAtNanos = System.nanoTime() +
TimeUnit.MILLISECONDS.toNanos(delayMillis);
        this.seq = SEQ.getAndIncrement();
    }

    @Override public long getDelay(TimeUnit unit) {
        long diff = runAtNanos - System.nanoTime();
        return unit.convert(diff, TimeUnit.NANOSECONDS);
    }
    @Override public int compareTo(Delayed o) {
        if (this == o) return 0;
        DelayedTask other = (DelayedTask) o;
        int c = Long.compare(this.runAtNanos, other.runAtNanos);
        return (c != 0) ? c : Long.compare(this.seq, other.seq);
    }
    @Override public String toString() { return id; }
}

public class Scheduler {
    private final DelayQueue<DelayedTask> queue = new DelayQueue<>();

    public void schedule(String id, long delayMs) {
        queue.put(new DelayedTask(id, delayMs));
```

```
    }

    public void startWorker() {
        Executors.newSingleThreadExecutor().submit(() -> {
            try {
                while (true) {
                    DelayedTask t = queue.take(); // blocks until due
                    System.out.println("Running: " + t);
                    // execute task...
                }
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        });
    }

    public static void main(String[] args) {
        Scheduler s = new Scheduler();
        s.startWorker();
        s.schedule("email-reminder", 500);
        s.schedule("retry-payment", 1500);
    }
}
```

Reasoning to interviewer

- DelayQueue centralizes timing; no need for per-task timers or sleeps.
- Guarantees the next take() only returns when that item is due.
- Great for retry backoff and scheduled notifications.

Future scope

- For cron-like schedules, pair with ScheduledThreadPoolExecutor.
- Persist scheduled jobs to survive restarts; on boot, rehydrate into the queue.
- Support cancellation by tracking IDs and removing from the queue when needed.

Scenario 73: Sorted, concurrent map with range queries

You maintain price points (or time-series) that must be accessible concurrently and support range queries (e.g., all prices between 100 and 150) with predictable ordering.

Answer

Use ConcurrentSkipListMap<K, V>.

- A lock-free, sorted, concurrent map (skip-list based).
- Supports headMap, tailMap, subMap, firstEntry, lastEntry under concurrency.
- Predictable iteration order without global locks.

Code

```
import java.util.NavigableMap;
import java.util.concurrent.ConcurrentSkipListMap;

public class PriceBook {
    private final ConcurrentSkipListMap<Integer, String> prices = new
ConcurrentSkipListMap<>();

    public void put(int price, String product) { prices.put(price, product); }
    public String at(int price) { return prices.get(price); }
    public NavigableMap<Integer, String> between(int low, int high) {
        return prices.subMap(low, true, high, true);
    }

    public static void main(String[] args) {
        PriceBook book = new PriceBook();
        book.put(99, "Mouse");
        book.put(149, "Keyboard");
        book.put(129, "Headset");

        System.out.println("All (sorted): " + book.prices);
        System.out.println("Range [100..150]: " + book.between(100, 150));
        System.out.println("Best (lowest): " + book.prices.firstEntry());
        System.out.println("Best (highest): " + book.prices.lastEntry());
    }
}
```

Reasoning to interviewer

- ConcurrentHashMap is fast but unordered; can’t do range queries without extra structures.
- TreeMap has ordering and range ops but isn’t thread-safe.

- ConcurrentSkipListMap combines both: ordering + concurrency + navigable range views.

Future scope

- For time-series, use timestamps as keys; snapshot via subMap(start, end).
- Use descendingMap() for reverse scans (e.g., latest-first).
- If values are large, store IDs and fetch payloads from a cache or DB to reduce map pressure.

Scenario 74: Zero-buffer handoff between threads (rendezvous)

You're wiring a pipeline where producers must hand a task directly to a worker; if no worker is ready, the producer waits. No queueing/buffering is desired.

Answer

Use a SynchronousQueue<Task>.

- Capacity is effectively zero: each put() must pair with a take().
- Great for throttling/handing off work when you want strict producer–consumer rendezvous.
- Avoids internal buffering; backpressure is immediate.

Code

```
import java.util.concurrent.*;

class Task { final String id; Task(String id) { this.id = id; } }

public class DirectHandoff {
    public static void main(String[] args) throws Exception {
        SynchronousQueue<Task> handoff = new SynchronousQueue<>();

        // Consumer
        Thread worker = new Thread(() -> {
            try {
                while (true) {
                    Task t = handoff.take(); // blocks until a producer arrives
                    System.out.println("Processing " + t.id);
                }
            } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
        });

        worker.start();

        // Producer (blocks until worker takes)
        handoff.put(new Task("T-1"));
        handoff.put(new Task("T-2"));
    }
}
```

Reasoning to interviewer

- Unlike LinkedBlockingQueue, this enforces a strict rendezvous—no build-up of tasks.

- Excellent when the cost of producing should be naturally paced by consumption capacity.
- Also useful for “work handoff” where you want immediate processing, not buffering.

Future scope

- For multiple workers, pair with a fixed thread pool; producers will match whichever worker is ready.
- If you want time-bounded handoff, use offer(task, timeout, unit) and handle timeouts.
- Switch to a bounded BlockingQueue when you need small buffers instead of zero.

Scenario 75: Lock-free multi-producer/multi-consumer event bus

You need a highly concurrent, low-latency FIFO mailbox where many threads publish events and many threads consume them; no blocking required.

Answer

Use ConcurrentLinkedQueue<Event> (or ConcurrentLinkedDeque if you need both ends).

- Lock-free (CAS-based), non-blocking MPMC queue.
- Iteration is weakly consistent—safe under concurrent modifications.
- Perfect for telemetry, async logging, lightweight mailboxes.

Code

```
import java.util.Queue;
import java.util.concurrent.ConcurrentLinkedQueue;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

class Event { final String msg; Event(String msg) { this.msg = msg; } }

public class EventBus {
    private final Queue<Event> queue = new ConcurrentLinkedQueue<>();

    public void publish(Event e) { queue.offer(e); }

    public void startConsumers(int n) {
        ExecutorService pool = Executors.newFixedThreadPool(n);
        for (int i = 0; i < n; i++) {
            pool.submit(() -> {
                while (!Thread.currentThread().isInterrupted()) {
                    Event e = queue.poll();         // non-blocking
                    if (e != null) {
                        System.out.println("Handled: " + e.msg);
                    } else {
                        Thread.onSpinWait();        // or small sleep/backoff
                    }
                }
            });
        }
    }

    public static void main(String[] args) {
        EventBus bus = new EventBus();
        bus.startConsumers(2);
        for (int i = 1; i <= 5; i++) bus.publish(new Event("E" + i));
    }
```

```
}
```

Reasoning to interviewer

- BlockingQueue is great for coordination, but here we prefer non-blocking and maximum concurrency.
- ConcurrentLinkedQueue scales well under many producers/consumers with minimal contention.
- Caveat: without blocking, you need a backoff or batching strategy to avoid busy-wait CPU burn.

Future scope

- If head/tail operations from both ends are needed, switch to ConcurrentLinkedDeque.
- For backpressure, move to a bounded BlockingQueue or add a semaphore on offer.
- Batch poll() in loops to amortize contention and improve cache locality.

Scenario 76: Memory-sensitive cache that won’t prevent GC

You cache computed thumbnails keyed by file path, but don’t want the cache to keep memory from being reclaimed under pressure.

Answer

Use WeakHashMap<String, SoftReference<byte[]>>.

- Weak keys: if no strong ref to the key exists elsewhere, the entry can be reclaimed.
- Soft values: values are retained longer than weak references but may be cleared under memory pressure.
- Together, the cache shrinks automatically when memory is tight without leaks.

Code

```
import java.lang.ref.SoftReference;
import java.util.Map;
import java.util.WeakHashMap;

public class ThumbCache {
    private final Map<String, SoftReference<byte[]>> cache = new
WeakHashMap<>();

    public void put(String path, byte[] thumb) {
        cache.put(path, new SoftReference<>(thumb));
    }

    public byte[] get(String path) {
        SoftReference<byte[]> ref = cache.get(path);
        return ref == null ? null : ref.get(); // may have been GC'd
    }

    public static void main(String[] args) {
        ThumbCache c = new ThumbCache();
        c.put("/img/a.png", new byte[1024]);
        byte[] v = c.get("/img/a.png");
        System.out.println("Present? " + (v != null));
        // Later, under memory pressure, the SoftReference may be cleared → get()
returns null
    }
}
```

Reasoning to interviewer

- WeakHashMap alone uses weak keys; values remain strongly referenced (not ideal for big payloads).
- SoftReference values allow GC to reclaim heavy objects when needed, while still acting as a cache.
- Great when recomputation is acceptable and you want to avoid OOM from caching.

Future scope

- For production-grade caches (size/time eviction, stats), prefer Caffeine or Guava Cache over raw references.
- If keys themselves are always strongly referenced elsewhere (e.g., canonicalized strings), you can use a normal HashMap<String, SoftReference<V>>.
- Add a cleanup step to remove entries whose SoftReference has been cleared (null value) during reads/writes.